MossOS

设计文档

参赛院校：重庆邮电大学

参赛队名：每天睡满八小时

队伍成员：吴勋、张可欣

2026 年 6 月

目录

1 概述 1
1.1 项目简介 1
1.2 系统架构 1
1.3 初赛成果 2
1.4 代码结构 3
2 进程与线程管理 4
2.1 概述 4
2.2 核心设计思想 4
2.3 关键数据结构 5
2.4 进程与线程创建 6
2.5 程序执行 8
2.6 退出与资源回收 9
2.7 进程组与会话 10
2.8 权限与进程控制 11
2.9 本章小结 12
3 调度与同步机制 13
3.1 概述 13
3.2 任务调度机制 14
3.3 阻塞与唤醒机制 17
3.4 通用阻塞机制的应用 20
3.5 Futex 同步机制 21
3.6 本章小结 23
4 内存管理 24
4.1 概述 24
4.2 栈自动增长 25
4.3 mremap 迁移 26
4.4 MADV_WIPEONFORK 26
4.5 /proc/[pid]/maps 26
4.6 PROT_WRITE 与 SIGBUS 26
4.7 COW 与缺页路径的改进 27
4.8 本章小结 27
5 文件系统 28
5.1 概述 28
5.2 挂载子系统 29
5.3 inode 元数据缓存 31
5.4 扩展属性 32
5.5 文件事件通知 32
5.6 I/O 性能优化 34
5.7 本章小结 37
6 信号机制 38
6.1 StarryOS 的信号机制 38
6.2 进程可暂停状态 38
6.3 信号驱动的 wait 语义 40
6.4 父进程如何感知状态变化 41
6.5 进程身份与信号权限校验 42
6.6 小结 42

7 网络模块 43
7.1 StarryOS 的网络栈 43
7.2 地址族支持与兼容性补齐 43
7.3 Buffer 扩展 45
7.4 Loopback 路径优化 45
7.5 性能对比 47
7.6 小结 49
8 QEMU 虚拟硬件平台与设备驱动 51
8.1 平台抽象层架构 51
8.2 双架构 QEMU 平台对比 52
8.3 VirtIO 设备驱动 52
8.4 跨平台行为一致性 54
9 总结与展望 55
9.1 工作总结 55
9.2 未来工作 55

第 1 章 概述

1.1 项目简介

MossOS 是用 Rust 语言实现的宏内核，基于 ArceOS 的组件化框架，从 StarryOS 演化而来。ArceOS 把内核功能拆成独立的 crate：硬件抽象层、调度器、内存分配器、文件系统框架和网络栈各自独立，按需组合。MossOS 在这套底层之上实现进程管理、信号处理、文件描述符抽象和系统调用，构成完整的宏内核。

目前 MossOS 支持 RISC-V 64 和 LoongArch64，两种架构共享同一套内核源码，硬件差异由 ArceOS 底层屏蔽。

1.2 系统架构

MossOS 的整体结构见 图 1-1，从上到下分为五层。

用户态程序通过系统服务接口层（syscall 入口）进入内核。内核主体按功能域分为八个子系统：任务与执行管理覆盖进程和线程的完整生命周期；地址空间与内存管理负责用户地址空间；文件与存储管理提供统一的文件接口；I/O 与事件通知处理异步 I/O 和多路复用；IPC 与同步机制处理进程间通信；网络通信子系统实现网络协议栈；系统资源管理管理进程凭据与资源约束；内核运行支撑提供基础运行时设施。

内核主体之下是 ArceOS 的组件化基础设施。ArceOS 将内核功能拆解为独立的 crate：`axtask` 提供任务管理，`axmm` 处理内存分配，`axfs` 实现文件系统框架，`axnet` 封装网络协议栈，`axhal` 定义硬件抽象接口，`axsync` 提供同步原语，`axruntime` 负责运行时初始化，`axplat` 负责平台抽象层。MossOS 从 StarryOS 继承而来，StarryOS 已在这些组件基础上构建了宏内核功能，MossOS 沿用这套架构并在此之上进行了完善。

其中硬件抽象分三层实现。最上层 `axhal` 向内核其余部分提供统一的硬件抽象接口。`axhal` 收到请求后调用平台抽象层 `axplat` 的函数，根据启动配置转发到具体架构的平台抽象层（`axplat-riscv64-qemu-virt` 对应 RISC-V 和 `axplat-loongarch64-qemu-virt` 对应 LoongArch64）进行进一步处理。最终调用最底层 `axcpu` 里的函数，使用汇编语言对 CPU 寄存器进行修改。

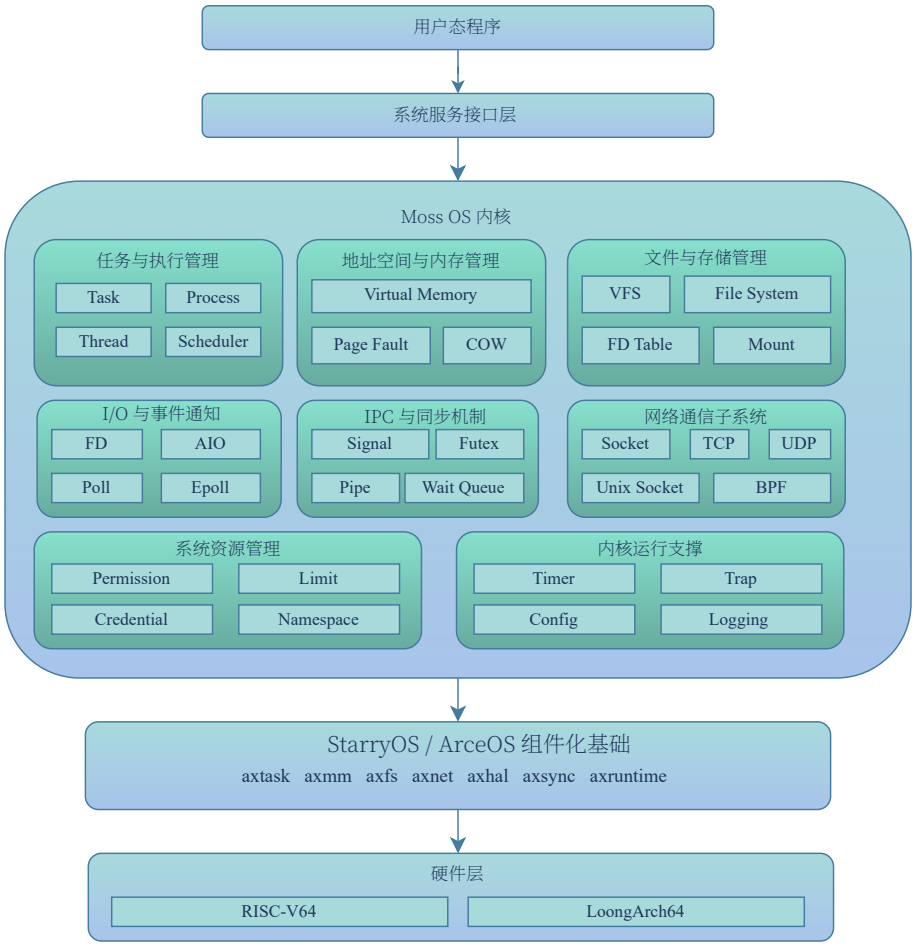


图 1-1　MossOS 系统整体架构

1.3 初赛成果

截至 6 月 30 日 20:22，MossOS（队名“每天睡满八小时”）在初赛排行榜上位列第 3，综合得分 2601.18。进程管理、内存管理、文件系统、信号、网络和设备驱动几个主要模块已具备主干功能，系统调用覆盖了 Linux ABI 的常用接口。在 QEMU 上可以运行 busybox shell、lua 和 libc 测试程序。

#	用户名	队伍	学校	提交次数 (ASC)	最后提交时间 (ASC)	basic-glibc-la	basic-glibc-rv	basic-musl-la	basic-musl-rv	busybox-glibc-la	busybox-glibc-rv	busybox-musl-la	bu mu
1	T202610486999845	alREDy	武汉大学	262	2026-06-28 16:24:33	100.0	102.0	102.0	102.0	54.0	54.0	54.0	54.
2	T202610459999616	PulseOS	郑州大学	164	2026-06-29 11:25:21	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.
3	T202610617999569	每天睡满八小时	重庆邮电大学	200	2026-06-24 02:43:21	102.0	102.0	102.0	102.0	54.0	54.0	54.0	54.

图 1-2　初赛排行榜（截至 2026 年 6 月 30 日）

1.4 代码结构

```
MossOS/
├── src/
│   └── main.rs              # 内核入口，初始化运行时
├── kernel/
│   └── src/
│       ├── task/            # 进程与线程管理
│       ├── mm/              # 地址空间与 ELF 加载
│       ├── file/            # FileLike 抽象与文件描述符表
│       ├── pseudofs/        # procfs / sysfs 伪文件系统
│       ├── syscall/         # 系统调用分发与实现
│       └── config/          # 内核配置
├── vendor/                  # 定制版 ArceOS 组件
│   ├── axhal/               # 硬件抽象层
│   ├── axfs/ axfs-ng-vfs/   # 文件系统框架
│   ├── axplat-riscv64-qemu-virt/     # RISC-V 64 平台实现
│   ├── axplat-loongarch64-qemu-virt/ # LoongArch64 平台实现
│   ├── starry-process/      # 进程抽象
│   ├── starry-signal/       # 信号库
│   └── starry-smoltcp/      # 网络库
├── third_party/             # 第三方开源库
│   ├── lwext4_rust          # ext4 文件系统支持库
│   └── syscalls             # 系统调用常量与类型定义
├── docs/                    # 设计文档
└── Makefile                 # 构建入口
```

`vendor/` 里是 ArceOS 上游组件的定制版，接口与上游保持兼容，按宏内核需要做了扩展。各模块的设计在后续各章展开。

第 2 章 进程与线程管理

进程与线程管理负责组织用户程序的执行主体及其资源，向内连接地址空间、文件描述符和信号等内核对象，向外提供创建、执行、退出、等待和控制接口。

2.1 概述

StarryOS 已经建立了可运行的进程线程框架。每个用户执行流对应一个 `Thread`，同一进程中的线程共同持有 `ProcessData`。前者保存线程私有状态，后者保存地址空间、文件资源、进程级信号和父子关系等共享状态。在功能上，StarryOS 已经提供了 `fork`、`clone`、`clone3`、`execve`、`exit` 和 `waitpid`，能够完成进程创建、程序映像替换、线程退出和僵尸进程回收，进程组、会话和 pidfd 也已有基本实现。

然而，基线只处理了“创建子进程、运行、退出、父进程回收”这条主路径，接口组合复杂后问题便逐渐暴露：clone 的标志约束和失败回滚不够严密，exec 缺少 `execveat` 及执行权限检查，wait 无法完整表达停止和继续状态，UID/GID 部分接口直接返回成功而没有保存身份变化，ptrace 和 pidfd 的观测链路也尚未打通。换句话说，基线能建立进程，却难以把进程的身份、状态和生命周期完全串联起来。

MossOS 保留了 StarryOS 的主体结构，重点补齐这些接口之间的联系。主要增量如表所示：

内容	StarryOS 基线	MossOS 的增量
对象与状态	`Thread` 与 `ProcessData` 分离	补充凭据、停止/继续状态、子进程时间和 namespace 元数据
创建与执行	`clone3`、`execve` 已实现	补严 clone ABI 和失败回滚，增加 `execveat` 与执行约束
退出与等待	线程退出、僵尸进程和 `waitpid`	完善资源清理，增加 `waitid`、状态事件和 pidfd 等待
管理接口	进程组、会话、基础凭据与 pidfd	修正全局登记，实现有状态凭据和最小 ptrace 控制链路

表 2-1　进程与线程模块的主要增量

2.2 核心设计思想

StarryOS 已经搭起了进程线程管理的基本框架，MossOS 的工作是在这套框架中进一步理清状态归属和对象关系：哪些数据属于单个线程，哪些资源应由整个进程共享；进程之间如何组织；身份信息怎样参与权限判断；一个任务又该如何从创建走到退出和回收。下面从这几个问题分别说明。

2.2.1 双层对象模型

进程和线程并不是两套互不相干的对象。MossOS 延续 StarryOS 的做法，将状态分为两层：`Thread` 表示单个执行流，`ProcessData` 表示线程共同拥有的进程资源。创建普通线程时复用 `ProcessData`；创建新进程时再根据 clone 标志复制或共享其中的资源。

这种划分回答了一个最基本的问题：某项状态究竟属于线程，还是属于整个进程。线程名称、线程信号状态和退出字段放在 `Thread` 中；地址空间、程序路径、文件表作用域、凭据和子进程集合放在 `ProcessData` 中。只有边界清楚，clone、exec 和 exit 才不会各自维护一份互相冲突的状态。

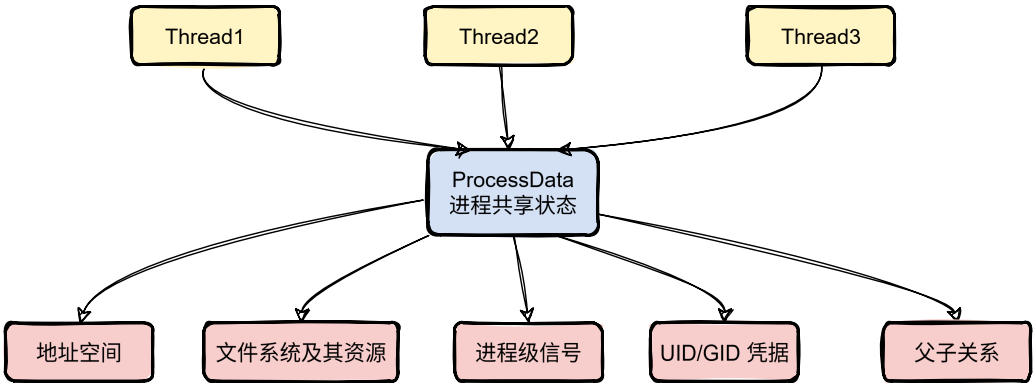


图 2-1　Thread 与 ProcessData 的双层对象模型

2.2.2　三类组织关系

MossOS 同时维护三类关系。

- 归属关系：多个线程通过共享 `ProcessData` 归属于同一进程。
- 亲缘关系：进程通过 parent 和 children 形成父子树，供退出托管和 wait 使用。
- 分组关系：进程依次归入进程组和会话，形成 PID、PGID、SID 三层标识。

这三类关系用途不同。线程组解决“哪些执行流共享资源”，父子树解决“谁负责回收谁”，进程组和会话解决“哪些进程作为一个作业被统一管理”。MossOS 没有把它们压进同一张表，而是分别维护，再通过全局索引提供查询。

2.2.3　三元凭据

MossOS 的进程身份设计参考了 Linux 的凭据模型：UID 不只保存一个值，而是拆成 real、effective、saved 三个字段，GID 采用相同结构。real ID 表示身份来源，effective ID 用于当前权限判断，saved ID 允许进程在受限范围内恢复先前身份。MossOS 将 UID/GID 三元组放入 `ProcessData`，让凭据真正参与程序执行、信号发送和 ptrace 权限检查。具体实现见 小节 2.8.1。

2.2.4　生命周期闭环

进程管理不是若干独立系统调用的集合，而是一条连续的生命周期：

创建 `clone` → 执行 `exec` → 退出 `exit` → 观察与回收 `wait`

创建阶段建立身份、资源和父子关系；exec 替换程序映像，但保留 PID、父子关系和进程组；exit 释放运行资源并留下退出状态；wait 读取状态并完成最终回收。MossOS 的多项修改都围绕这条闭环展开，尤其重视失败路径和中间状态，而不仅是系统调用成功时的主路径。

2.3　关键数据结构

2.3.1　Thread 与 ProcessData

`Thread` 和 `ProcessData` 是这部分最重要的两类数据，下面是关键代码片段。

```
pub struct Thread {
    // 共享进程级资源
    pub proc_data: Arc<ProcessData>,
    // 线程私有状态
    pub signal: Arc<ThreadSignalManager>,
    pub exit: Arc<AtomicBool>,
}
```

```rust
pub struct ProcessData {
    // 进程身份及组织关系
    pub proc: Arc<Process>,
    // 线程共享的进程资源
    pub aspace: Arc<Mutex<AddrSpace>>,
    pub scope: RwLock<Scope>,
    pub wait_status: SpinNoIrq<Option<ProcessWaitStatus>>,
    // UID/GID 三元组
    uid: AtomicU32, euid: AtomicU32, suid: AtomicU32,
    gid: AtomicU32, egid: AtomicU32, sgid: AtomicU32,
}
```

代码 2-1　线程私有状态与进程共享状态

同一进程中的线程持有同一个 `Arc<ProcessData>`，因此能看到相同的地址空间、文件资源和凭据。`Thread` 自身则保存只与当前执行流相关的数据。MossOS 新增字段时继续遵循这一边界，没有另建一套进程控制块。

2.3.2　四张全局索引表

内核需要按 TID、PID、PGID 和 SID 查找对象。MossOS 使用四张弱引用表分别索引线程、进程、进程组和会话：

索引表	键	作用
`TASK_TABLE`	TID	查找具体线程和线程私有状态
`PROCESS_TABLE`	PID	查找进程共享资源和父子关系
`PROCESS_GROUP_TABLE`	PGID	查找进程组并执行组级操作
`SESSION_TABLE`	SID	查找会话及其进程组层级

表 2-2　进程线程对象的全局索引

表中保存弱引用，不会因为全局索引而阻止对象释放。MossOS 还修正了新建进程组和会话后的登记过程，避免对象已经创建，却无法通过 PGID 或 SID 查到。

2.4　进程与线程创建

2.4.1　fork 与统一的 clone 路径

`fork` 在 MossOS 中并未缺席。在 x86_64 平台上，`sys_fork` 会调用 `sys_clone`，其中不设置资源共享标志，只指定子进程退出时发送 `SIGCHLD`。因此，`fork` 创建的是拥有独立 `ProcessData` 的子进程；`clone` 和 `clone3` 则允许用户通过标志决定创建进程还是线程，以及资源如何继承。

这些入口最终都把参数整理为 `CloneArgs`，交给 `CloneArgs::do_clone` 执行。带有 `CLONE_THREAD` 时，新任务复用当前 `ProcessData`；否则调用 `Process::fork` 建立子进程及父子关系，再创建新的 `ProcessData`。地址空间、文件表、文件系统上下文和信号处理动作分别由 `CLONE_VM`、`CLONE_FILES`、`CLONE_FS` 和 `CLONE_SIGHAND` 决定共享或复制。

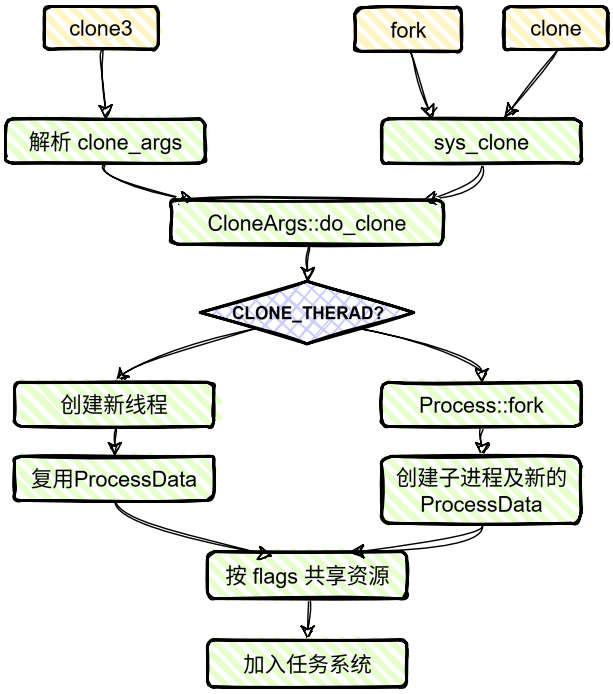


图 2-2　fork、clone 与 clone3 的统一创建路径

标志	作用
CLONE_THREAD	加入调用者的线程组，共享同一个 ProcessData
CLONE_VM	共享虚拟地址空间
CLONE_FILES	共享文件描述符表
CLONE_FS	共享文件系统上下文
CLONE_SIGHAND	共享信号处理动作表

表 2-3　进程线程创建中的主要 clone 标志

这些标志可以组合使用，但并非任意组合都合法。CLONE_THREAD 决定新任务是否仍属于原进程，其余标志主要控制资源如何继承，或指定创建前后的附加操作。

2.4.2　参数约束

MossOS 补充了 clone 标志之间的依赖关系，让不合法的参数组合直接报错，而不是创建一个运行后才暴露问题的异常线程/进程，把 bug 扼杀在摇篮里。例如，线程必须共享地址空间和信号处理动作，共享信号处理动作又必须共享地址空间：

```
// 线程必须共享地址空间和信号处理动作
if flags.contains(CloneFlags::THREAD)
    && !flags.contains(CloneFlags::VM | CloneFlags::SIGHAND)
{
    return Err(AxError::InvalidInput);
}
// 共享信号处理动作必须共享地址空间
if flags.contains(CloneFlags::SIGHAND)
    && !flags.contains(CloneFlags::VM)
{
    return Err(AxError::InvalidInput);
}
```

代码 2-2　clone 标志组合校验

对于 clone3，MossOS 进一步检查结构体最小长度、栈地址与栈大小的配对关系，以及超出当前结构体版本的尾部字节。额外字节必须可读且全部为零，使旧内核能够安全接收新版本用户态传入的结构。

2.4.3　创建失败回滚

这里处理的情况和 2.3.3 的不同。2.3.3 讨论的是子进程已经运行后，父进程怎样通过 wait 读取它的状态；如果子进程还没有开始运行，clone 就已经失败，那么系统调用会向调用者返回错误，但内核此前可能已登记 PID、父子关系和线程记录，必须把这些半成品全部撤销。

do_clone 会先建立进程对象和父子关系，分配 ProcessData，再创建线程记录并加入全局索引。若调用者请求 CLONE_PIDFD，内核还要分配文件描述符，并将其编号写回用户地址。只有这些步骤都完成后，新任务才会提交给调度器正式运行，如图 2-3 所示。

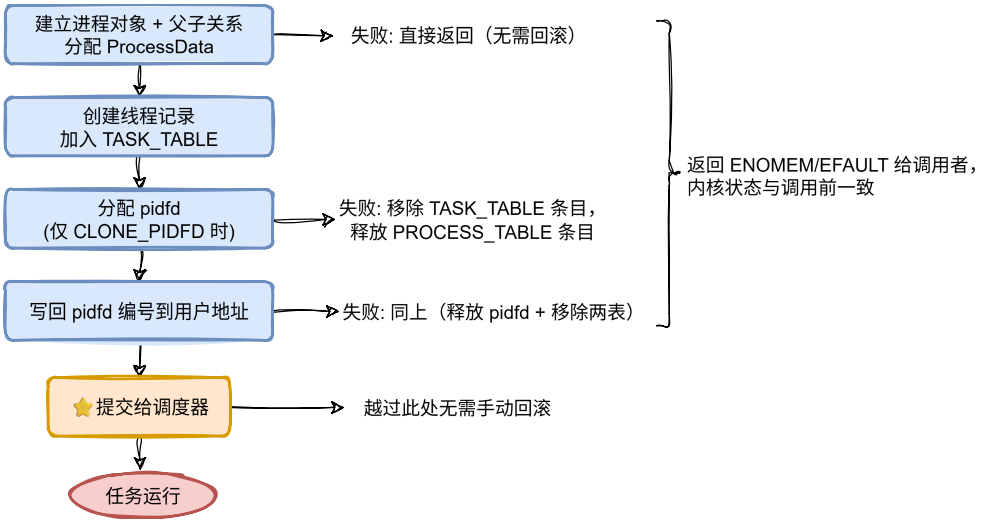


图 2-3　do_clone 执行步骤与失败回滚路径

pidfd 分配时文件描述符表可能已满，向用户地址写回 pidfd 编号时也可能遇到无效指针。这些错误发生时，进程对象已经登记进 PROCESS_TABLE，线程记录已经加入 TASK_TABLE，但新任务尚未运行，因此不会自行调用 exit() 进入僵尸状态。若直接返回错误，父进程会在 waitpid(-1, ...) 中等不到这个子进程的退出通知，PID 也会被永久占用；PROCESS_TABLE 中的条目还会因父进程 children 表的强引用而无法自动释放。

MossOS 因此在任务提交调度器之前统一处理失败路径：释放已分配的 pidfd，从全局索引移除线程记录；若新进程没有其他存活线程，再一并释放进程对象。这样 do_clone 返回错误时，内核状态与调用前一致，调用者看到的“创建失败”才真正表示这次创建没有留下任何痕迹。

2.5　程序执行

2.5.1　execve 基线

StarryOS 的 execve 已经能够装载 ELF 文件，替换地址空间和用户上下文，更新程序路径与命令行，重置信号处理动作，并关闭带有 CLOEXEC 标志的文件描述符。exec 不创建新进程，因此 PID、父子关系、进程组和会话保持不变。基线的不足在于只能按路径执行文件，对执行权限、MS_NOEXEC 挂载点和正在写入的可执行文件缺少检查，也没有 execveat。

2.5.2　execveat 与执行检查

传统 execve 只能接收路径，绝对路径依赖进程看到的文件系统，相对路径又依赖当前工作目录。程序若已经打开了可信目录或目标文件，仍需重新拼接并解析路径，期间目录项可能被改

名或替换。execveat 可以从稳定的目录文件描述符开始解析，或使用 AT_EMPTY_PATH 执行已经打开的文件，减少重复路径解析带来的竞态，也便于容器运行时、动态加载器等程序在受控目录中启动目标。

MossOS 增加该入口后，将 execve 与 execveat 汇入同一条 execve_common 路径，后续装载和状态更新仍由一套逻辑完成。在替换旧地址空间之前，MossOS 先完成执行条件检查：

```
// 挂载点禁止执行
if exec_loc.mount_flags() & MS_NOEXEC != 0 {
    return Err(AxError::PermissionDenied);
}
// 按有效 UID/GID 检查执行位
if !has_exec_permission(exec_loc, proc_data.euid(), proc_data.egid())? {
    return Err(AxError::PermissionDenied);
}
// 拒绝正在被写入的可执行文件
if is_text_busy(exec_loc) {
    return Err(AxError::from(LinuxError::ETXTBSY));
}
```

代码 2-3　程序执行前的约束检查

这些检查全部通过后，内核才装载新程序并提交地址空间。若解析、权限或装载失败，原程序仍可继续运行。成功后，内核还会清除旧程序遗留的线程退出指针，避免新地址空间继续引用失效地址。

2.6　退出与资源回收

2.6.1　单个线程退出与进程结束

一个进程可以包含多个线程，因此某个线程退出并不等于整个进程结束。内核先执行线程级退出清理，再把它从进程的线程集合中移除。清理内容包括 robust futex 和 clear_child_tid，其唤醒语义放在第三章的 futex 退出路径中说明。如果还有其他线程存活，共享的地址空间、文件表和 ProcessData 都继续保留。

只有最后一个线程退出时，内核才结束整个进程。此时进程进入僵尸状态，保存退出码，向父进程发送退出信号并唤醒正在等待的父进程。MossOS 还在这一阶段释放文件锁、租约、共享内存记录和 ptrace 关系。进程对象不会立即删除，要等父进程调用 wait 读取退出状态后才完成回收。

2.6.2　exit_group：结束全部线程

普通 exit 只结束调用线程，exit_group 则需要主动让进程中的全部线程退出。MossOS 先标记线程组正在整体退出，再向其他存活线程发送终止信号。各线程仍按上一节的路径完成清理，最后离开的线程负责执行进程级退出工作。这一区别对多线程程序尤其重要。

2.6.3　退出后的孤儿进程托管

进程结束后，还要处理它尚未退出的子进程。子进程不能随父进程一起删除——它们之后退出时需要有进程负责读取退出状态，否则将永远停留在僵尸状态占用内核资源。MossOS 沿用 StarryOS 的做法，把这些子进程从原父进程的 children 表中移出，改由 init 进程托管。如 图 2-4 所示，整个过程只修改父进程指针，子进程本身的 PID、PGID 和 SID 均保持不变，仍可继续运行。它们以后成为僵尸进程时，由 init 调用 wait 完成回收。

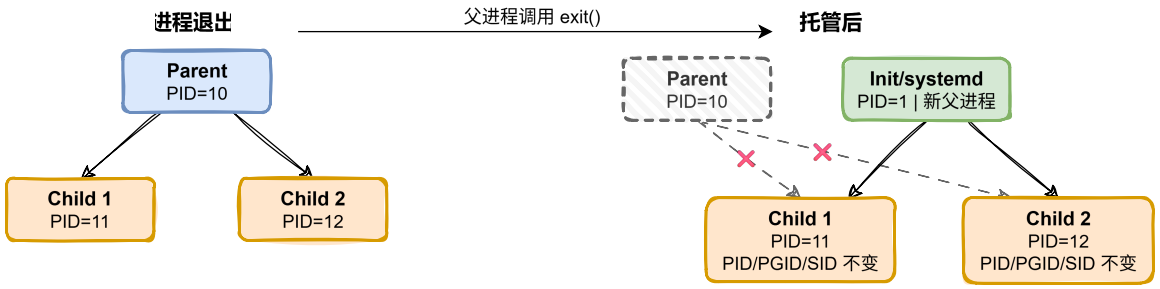


图 2-4　孤儿进程托管示意图

2.7　进程组与会话

2.7.1　PID、PGID 与 SID

进程组和会话用于组织多个相关进程。每个进程有 PID，所属进程组有 PGID，进程组所属会话有 SID。通常，组长的 PID 等于 PGID，会话首进程的 PID 等于 SID。这种层级常见于 Shell：一条管道中的多个进程属于同一进程组，一个终端登录环境中的多个进程组属于同一会话。它们仍是独立进程，但可以按组查询、等待或发送信号。

在实现上，MossOS 沿用 StarryOS 的对象关系：`Process` 保存自身 PID 和所属的 `ProcessGroup`，`ProcessGroup` 保存 PGID、所属 `Session` 及组内进程，`Session` 再保存 SID 和会话内的进程组。

```rust
pub struct Process {
    pid: Pid,
    // 每个进程只属于一个进程组
    group: SpinNoIrq<Arc<ProcessGroup>>,
}

pub struct ProcessGroup {
    pgid: Pid,
    // 进程组归属于一个会话
    session: Arc<Session>,
    // 记录组内的所有进程
    processes: SpinNoIrq<WeakMap<Pid, Weak<Process>>>,
}

pub struct Session {
    sid: Pid,
    // 一个会话可以包含多个进程组
    process_groups: SpinNoIrq<WeakMap<Pid, Weak<ProcessGroup>>>,
}
```

代码 2-4　进程、进程组与会话的对象关系

查询一个进程的三个标识时，内核沿对象引用逐层取得结果：先按 PID 找到进程对象，再沿所属关系取得进程组和会话标识。反向操作则依靠成员表完成——向进程组发送信号时，内核遍历 `ProcessGroup::processes`，逐个找到组内进程。

2.7.2 进程组创建与迁移

StarryOS 已提供 `setpgid`、`getpgid` 等基础接口，但新建的进程组没有同步写入全局 PGID 索引，导致其他系统调用可能无法通过 PGID 找到该进程组。

MossOS 根据目标 PGID 分两种情况处理。目标 PGID 等于进程自身 PID 时，内核创建新进程组并立即登记到全局索引，此后组信号和按进程组等待等操作都能查到该对象。目标 PGID 指向已有进程组时，内核先从全局索引中找到目标组，检查它与当前进程是否属于同一会话，通过后将进程从旧组迁移到新组，同时更新全局索引。若目标组不存在或属于其他会话，则拒绝迁移。这样修改后，进程所属组、成员表和全局索引始终保持一致。

2.7.3 会话创建

`setsid` 会同时创建新会话和新进程组。MossOS 在创建成功后分别登记 `Session` 和 `ProcessGroup`：

```
if let Some((session, group)) = proc.create_session() {
    // setsid 同时创建会话和进程组
    add_session_to_table(&session);
    add_process_group_to_table(&group);
    // SID 等于调用进程的 PID
    Ok(session.sid() as _)
}
```

代码 2-5　新会话与新进程组的创建和登记

2.8 权限与进程控制

2.8.1 UID/GID 三元组

进程的身份不能只用一个 UID 表示。系统既要知道它原本由谁启动，也要知道它此刻按谁的权限访问文件、执行程序或向其他进程发送信号。为此，MossOS 沿用 Linux 的三元凭据：real UID 记录原始用户，effective UID 用于当前权限检查，saved UID 保存此前可恢复的有效身份；GID 采用同样的结构。

这套设计用于受控的权限切换。普通用户运行带有 set-user-ID 属性的程序时，程序可以暂时以文件所有者的权限完成受保护操作，随后将 effective UID 降回普通用户。若后续仍需该权限，程序可从 saved UID 恢复，而不必始终以高权限运行。

StarryOS 基线中的部分身份设置接口只返回成功，并未保存身份变化；调用者看似完成了切换，后续权限检查却仍在使用旧身份。MossOS 在 `ProcessData` 中加入完整的六个 UID/GID 字段，并在 `setresuid`、`setresgid` 等接口中检查允许的转换范围：

```
let allowed = [old_ruid, old_euid, old_suid];
if old_euid != 0 {
    // 非特权进程不能把三元组改成从未拥有过的身份
    if !valid_id_change(ruid, &allowed)
        || !valid_id_change(euid, &allowed)
        || !valid_id_change(suid, &allowed)
    {
        return Err(eperm());
    }
}
```

```
// 分别更新三个字段，UID_GID_NO_CHANGE 表示不修改
if ruid != UID_GID_NO_CHANGE { proc_data.set_real_uid(ruid); }
if euid != UID_GID_NO_CHANGE { proc_data.set_euid(euid); }
if suid != UID_GID_NO_CHANGE { proc_data.set_suid(suid); }
```

代码 2-6　UID 三元组的保存与受限转换

普通进程不能凭空把自己改成另一个用户，只能在 real、effective 和 saved 三个已有值之间切换；effective UID 为 0 的进程才可以设置新的身份。effective UID/GID 一旦改变，后续权限检查也随之改变：进程可能失去执行受保护文件的权限，不能向其他用户的进程发送 `kill` 信号，或不能通过 ptrace 附加调试目标。凭据由此成为实际的权限依据，而不是查询接口返回的一组固定数字。

capability 则把传统上集中在 root 身上的权限拆成多项能力。MossOS 分别保存 effective、permitted、inheritable 和 bounding 集合：当前可使用的能力、允许启用的能力、可在执行新程序时传递的能力，以及能力的最高边界。更新这些集合时，内核会检查它们之间的约束，进程不能仅靠一次系统调用把原本没有的能力加入自身权限。

2.8.2　ptrace 最小控制链路

MossOS 新增 ptrace 的基础状态管理，建立 `TRACEME`、`ATTACH`、`DETACH`、停止、继续和 wait 报告之间的关系。attach 时先检查目标是否存在、是否已被跟踪以及调用者是否具有相应权限；目标停止后，跟踪者通过 wait 获取事件。

但是当前实现只能完成最小控制流程。`PTRACE_SYSCALL` 和 `PTRACE_SINGLESTEP` 暂按普通继续处理，寄存器、目标内存和 siginfo 读写请求返回 `EIO`，因此还不能视为完整的 GDB 后端。

2.9　本章小结

MossOS 选择沿用 StarryOS 的进程线程框架，围绕四条主线完成加固：以 `Thread` 和 `ProcessData` 组成双层对象模型，以线程归属、父子树和进程组会话维护三类组织关系，以 real、effective、saved ID 构成三元凭据，并把 clone、exec、exit、wait 串成完整生命周期。

在此基础上，MossOS 补严了 `clone3` 的 ABI 和失败回滚，增加 `execveat`，完善子进程状态、等待接口和退出清理，并加入有状态的权限和 ptrace 控制。不足之处在于当前还未完成完整 vfork、多线程 exec 时仅保留调用线程的收敛、终端作业控制（前台进程组、后台读写限制和孤儿进程组规则）、namespace 隔离和 ptrace 数据访问，但进程从创建到回收的主要管理关系已经比 StarryOS 基线更完整。

第 3 章 调度与同步机制

任务等待事件的过程看起来直接：检查条件、登记唤醒入口、进入阻塞，事件到了再恢复。但如果通知恰好在“条件检查完、Waker 还没登记”这个窗口里到达时，等待方错过了这次通知，之后就没有人再来唤醒它了。本章大多数改动都对准这个竞争窗口，以及由此派生出的几个具体问题：超时后旧等待项还留在队列里，子进程状态变化和父进程登记 Waker 之间有空隙，Futex 通知与入队之间存在同样的竞争。

MossOS 沿用 StarryOS 的 RR 调度框架，改动集中在进入阻塞和重新唤醒之间的那段路径。任务在就绪、运行和阻塞之间循环，执行完毕后退出调度流程，整体关系如图所示。

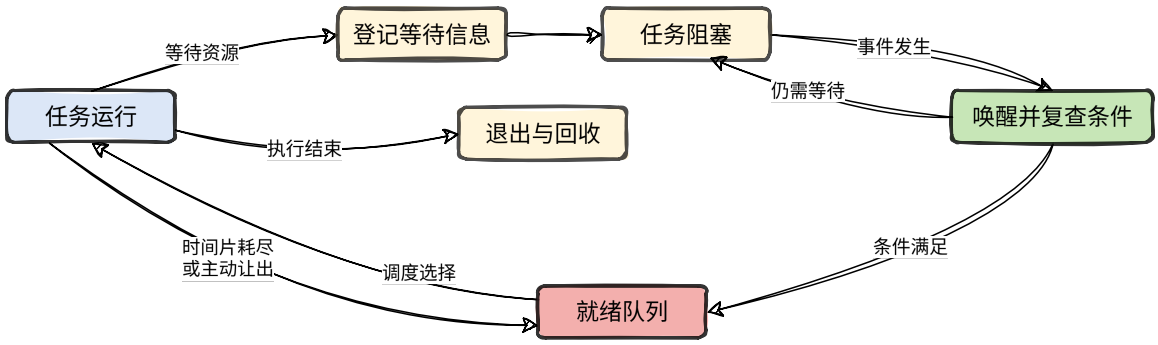


图 3-1　调度、阻塞与唤醒的总体循环

3.1 概述

StarryOS 已经提供了可运行的任务调度骨架：`axtask` 管理任务状态和运行队列，`axsched` 用 RR 调度器按时间片轮转，多核下每个 CPU 有独立就绪队列，Futex 的 WAIT/WAKE、位掩码唤醒、重新排队和 robust list 也已实现，定时器与子进程等待接口也在。这部分 MossOS 主要是在 StarryOS 的基础上进行补充的。

问题集中在事件交错时的状态处理上。Futex 等待项只保存了 Waker 和位掩码，没有独立的通知状态，超时或被信号中断后旧记录还留在队列里——内核说不清通知究竟有没有到达，失效等待项也会占用后续唤醒名额。`FUTEX_WAIT_BITSET` 的绝对截止时间被当相对时长用，定时器重新设置后旧事件照样留在全局队列。子进程状态变化与父进程登记 Waker 之间，也同样有一段通知可能悄悄溜走的窗口。

MossOS 保留 RR 调度框架，改动集中在三个环节：等待者如何登记，通知如何与阻塞状态配合，等待结束后如何清理记录。对应关系如下：

内容	StarryOS 基线	MossOS 的增量
任务调度	任务状态、RR 时间片和 CPU 亲和性	沿用调度框架，修正亲和性位图解析
阻塞唤醒	Future、Waker 和等待队列	调整 PollSet 唤醒过程，补充条件复查和等待项清理
定时处理	基础休眠和进程定时器	区分相对与绝对时间，使用 TimerKey 过滤旧事件
Futex	WAIT、WAKE、位掩码和重新排队	加入独立通知状态，修正超时语义和退出清理

表 3-1　调度与同步机制的主要增量

这些修改共同作用于任务离开 CPU 和重新获得 CPU 的过程。其中最难处理的是通知与状态转换可能同时发生：任务检查完条件、尚未登记 Waker 时，事件可能已经到达；任务准备进

入阻塞时，通知也可能先一步发生。下面的叙述会先用任务状态、抢占和上下文切换说明调度基础，再围绕这一竞争窗口讨论通用阻塞唤醒机制，最后分析定时等待、子进程等待和 Futex 的具体处理。

3.2 任务调度机制

3.2.1 任务状态与状态转换

任务调度围绕四种状态展开。调度器不需要理解任务正在等待哪一种资源，只需判断任务当前属于以下四种中的哪种状态：

```
pub enum TaskState {
    Running = 1, // 正在 CPU 上执行
    Ready   = 2, // 可以运行，正在队列中等待
    Blocked = 3, // 正在等待事件，暂时不能运行
    Exited  = 4, // 已经结束，等待资源回收
}

pub struct TaskInner {
    // 调度器通过该字段判断任务当前能否运行
    state: AtomicU8,
    // ...
}
```

代码 3-1　任务状态的数据结构

`state` 使用原子整数保存任务状态。定时等待、Futex 和子进程等待各自维护不同的业务条件，但要重新获得 CPU，都必须让任务从 `Blocked` 回到 `Ready`。调度器只处理这一状态转换，并不判断具体等待条件是否已经满足。四种状态之间的转换关系如图所示。

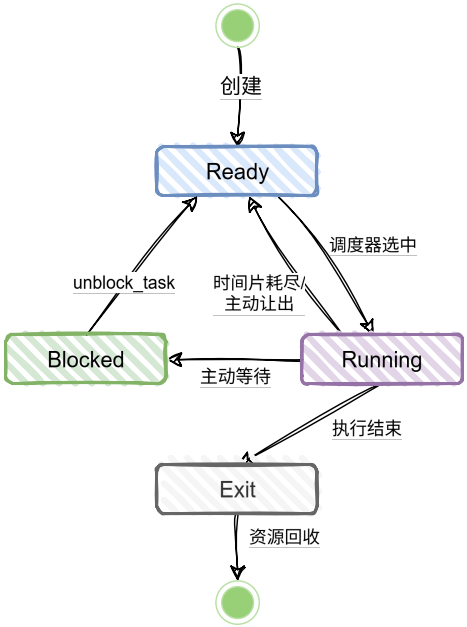


图 3-2　TaskState 的状态转换关系

新任务先以 `Ready` 状态进入就绪队列，被调度器选中后变为 `Running`。主动让出 CPU 的任务仍具备运行条件，因此会回到 `Ready`；等待资源的任务则进入 `Blocked`，由 `unblock_task` 恢复为

`Ready`。任务执行结束后进入 `Exited`，并在资源回收后离开状态机。MossOS 的定时等待、Futex 和子进程等待都通过这组状态转换重新接入调度器。

状态修改通过原子状态和运行队列中的唤醒检查配合完成。多个通知同时到达时，唤醒路径应该只允许仍处于 `Blocked` 的任务重新进入就绪队列。

3.2.2 时间片轮转与抢占

任务按照 RR 策略轮流使用 CPU。多核环境下，每个 CPU 都有自己的就绪队列，用来保存处于 `Ready` 状态的任务。调度器从队头选择下一个任务，任务的默认时间片是 5 个时钟滴答；时间片用完后，任务回到队尾，CPU 转交给后面的任务。

时钟中断每到来一次，`scheduler_timer_tick`（调度器的时钟节拍处理函数）就更新当前任务的时间片。计数归零时，内核先设置待抢占标志，而不是立刻在中断处理或持锁位置切换任务。等代码离开禁止抢占的临界区后，再执行真正的重新调度。

时钟中断 → 时间片减一 → 设置待抢占标志 → 离开临界区 → 重新调度

时间片归零后，内核先记录抢占请求，等执行离开禁止抢占的临界区再调度。任务如果在修改运行队列、处理中断或持有关键锁时被换出，其他 CPU 可能看到尚未完成的中间状态。延迟抢占把上下文切换放到状态一致的位置，运行队列和锁的处理也能先行结束。

如果当前没有普通任务可运行，CPU 会进入 idle 任务（没有普通任务时运行的空闲任务），等待新的任务或中断到来。

3.2.3 任务选择与上下文切换

前两节已经说明了任务状态如何变化，以及时间片耗尽后如何发起抢占。本节从重新调度真正开始的位置继续：当前任务的状态处理完毕后，`resched` 从本 CPU 的就绪队列中选出下一个任务；如果队列为空，就让 idle 任务暂时接管 CPU。

```rust
fn resched(&mut self) {
    // 从就绪队列选择任务；队列为空时运行 idle 任务
    let next = self.scheduler.lock().pick_next_task()
        .unwrap_or_else(|| unsafe {
            IDLE_TASK.current_ref_raw().get_unchecked().clone()
        });

    self.switch_to(crate::current(), next);
}

fn switch_to(&mut self, prev: CurrentTask, next: AxTaskRef) {
    // 将选中的任务设为正在运行
    next.set_state(TaskState::Running);

    // 保存当前任务现场，恢复下一个任务现场
    let prev_ctx = prev.ctx_mut_ptr();
    let next_ctx = next.ctx_mut_ptr();
    unsafe { (*prev_ctx).switch_to(&*next_ctx) };
}
```

代码 3-2　从选择任务到切换现场的主要流程

选出任务后，switch_to 先将它标记为 Running，再切换任务现场。这里的“现场”是任务暂停前的运行位置。每个任务都有一个 TaskContext，其中保存内核栈指针、需要保留的寄存器和返回位置。切换时，当前 CPU 的这些值被保存到旧任务中，再从新任务的 TaskContext 恢复。旧任务以后再次被选中时，就能接着原来的位置运行。

如 图 3-3 所示，两个任务的 TaskContext 分别位于各自的内核栈中，switch_to 把 prev 的寄存器值写出，再从 next 读入；其中 satp 让切换完成的瞬间就生效了新的地址空间。

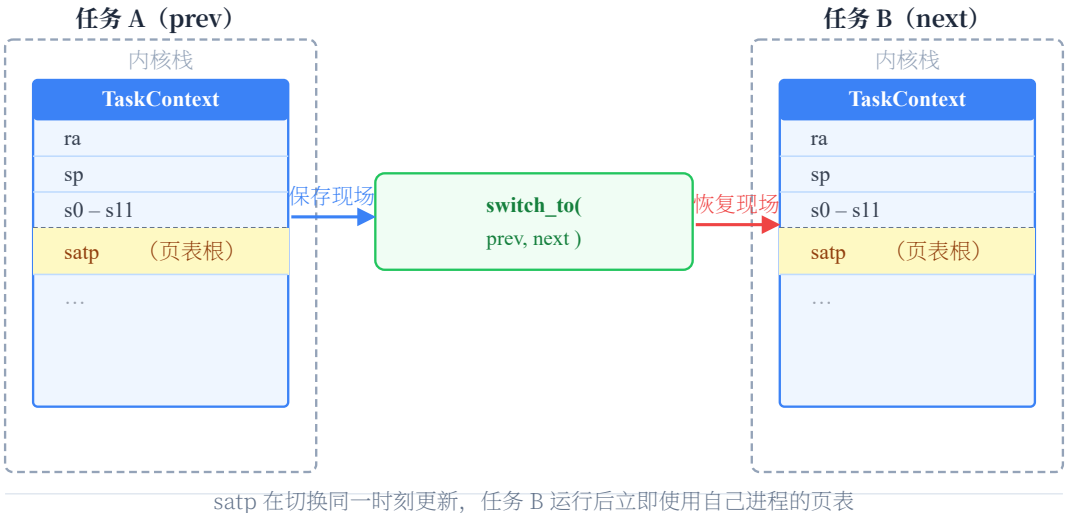


图 3-3　TaskContext 布局与 switch_to 的现场传递方向

对于用户任务，MossOS 在创建任务时把该进程的页表根写入 TaskContext，保证切换后使用正确的地址空间。用户态寄存器则保存在 UserContext（用户程序暂停时的寄存器记录）中。至此，从 3.2.2 的抢占请求到完成切换，可以连成下面这条流程：

设置待抢占标志 → 到达安全调度位置 → 选择下一任务 → 保存旧现场 → 恢复新现场

3.2.4　CPU 亲和性与调度接口

每个任务都保存一份 CPU 亲和性掩码 cpumask，用来限制它可以在哪些 CPU 上运行。例如，掩码只包含 CPU 0 和 CPU 2 时，任务就不会进入其他 CPU 的就绪队列。

新任务默认可以使用所有已经启用的 CPU。任务创建或被唤醒时，调度器在 cpumask 允许的范围内轮转选择一个 CPU，再把任务加入该 CPU 的就绪队列。

```
fn select_run_queue_index(cpumask: AxCpuMask) -> usize {
    loop {
        // 在所有 CPU 编号之间轮转
        let index = RUN_QUEUE_INDEX.fetch_add(1, Ordering::SeqCst)
            % axconfig::plat::CPU_NUM;
        // 只选择亲和性掩码允许的 CPU
        if cpumask.get(index) {
            return index;
        }
    }
}
```

代码 3-3　在亲和性范围内选择 CPU

这段逻辑按 CPU 编号轮转，并跳过亲和性掩码不允许的 CPU。它沿用原有的多运行队列设计，MossOS 在这一部分的实际修改主要位于用户位图解析和设置后的任务迁移。

用户传入的亲和性范围是一张位图，每一位对应一个 CPU。MossOS 修正了这张位图的解析过程：内核按照实际配置的 CPU 数量读取内容，再把其中被置为 1 的位置写入任务掩码。用户程序通过 Linux 的 `sched_getaffinity` 查询范围，通过 `sched_setaffinity` 修改范围。

```
// 用户位图中的每一位对应一个 CPU
for cpu_id in 0..axconfig::plat::CPU_NUM {
    if user_mask[cpu_id / 8] & (1 << (cpu_id % 8)) != 0 {
        cpu_mask.set(cpu_id, true);
    }
}

// 保存新的亲和性范围
let curr = current().clone();
curr.set_cpumask(cpu_mask);

// 当前 CPU 已不在范围内时，将任务迁移到允许的 CPU
if !cpu_mask.get(axhal::percpu::this_cpu_id()) {
    let migration_task = TaskInner::new(
        move || migrate_entry(curr),
        "migration-task".into(),
        4096,
    ).into_arc();
    current_run_queue().migrate_current(migration_task);
}
```

代码 3-4　CPU 亲和性掩码的设置与任务迁移

新的掩码保存后还要检查任务当前所在的 CPU。如果该 CPU 仍被允许，任务可以继续运行；如果已经被排除，MossOS 会把任务迁移到另一个允许的运行队列。新的设置因此会立即改变任务实际运行的位置，而不只是留下一个供查询的数值。

用户调用 `sched_yield` 主动让出 CPU 后，MossOS 将当前任务放回就绪队列，再由调度器选择其他任务运行。这个接口适合任务暂时没有紧急工作、希望让其他就绪任务先执行的情况。

3.3　阻塞与唤醒机制

MossOS 延续 StarryOS 的 Future/Waker 模型。这套机制的实际好处在于：timerfd、pidfd、Futex 等来源各不相同的等待，都能挂到同一套唤醒路径上，`block_on` 也因此能以统一方式暂停和恢复任务，不需要为每类等待单独实现阻塞逻辑。代价是每个等待路径都得正确维护通知状态和双重检查，否则竞争窗口就会漏掉通知。这正是本节要讨论的。

3.3.1　事件等待的基本流程

调度器只知道任务处于 `Blocked`，并不知道任务在等什么。事件对象和等待者之间还需要一套通知协议。MossOS 沿用 Future 表示”尚未完成、之后还要继续检查的操作”，用 Waker 记录”条件可能变化时如何通知当前任务”。

以读取管道为例，任务先检查缓冲区。已有数据就直接读取；没有数据则把自己的 Waker 登记到管道中，再由 Future 返回“暂时未完成”。另一个任务写入数据后，管道调用保存的 Waker，通知读任务继续检查。

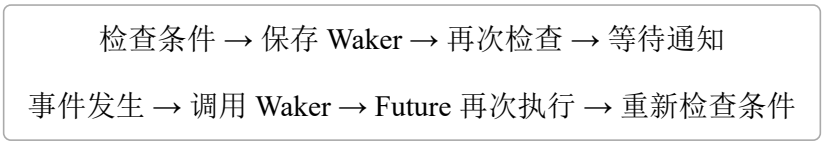


图 3-4　事件登记与通知的基本流程

关键步骤是登记 Waker 后的第二次条件检查。假如数据恰好在第一次检查结束后到达，此时写入方还找不到等待者，无法调用 Waker。登记完成后立刻复查缓冲区，读任务仍能看到已经到达的数据，不会因为漏掉那次通知而一直等待。

收到通知后还要重新检查条件。Waker 只表示“情况可能有变化”，并不保证资源一定属于当前任务。例如多个读任务同时被通知时，管道中的数据可能已经被其中一个任务取走，其他任务仍需继续等待。

3.3.2　Waker 与 PollSet

Waker 是 StarryOS 留给事件对象的任务唤醒入口。事件对象不必直接操作运行队列，只需保存等待者提供的 Waker，并在状态发生变化时调用 `wake()`。MossOS 继续使用这一接口，把新增的 timerfd、pidfd 和 Futex 等等待路径接到同一套任务唤醒流程上。

`PollSet` 管理一组 Waker，管道、eventfd、timerfd、pidfd 和终端等对象都通过它登记等待者。MossOS 增加 `has_waiters`，没有等待者时直接返回；真正唤醒时，先在锁内取出并清空本轮 Waker，再到锁外逐个调用。内部锁只保护集合操作，不覆盖可能继续触发调度的 `wake()`。

```
pub struct PollSet {
    inner: Lazy<Mutex<Inner>>,
    // 没有等待者时，wake 可以直接返回
    has_waiters: AtomicBool,
}

pub fn wake(&self) -> usize {
    if !self.has_waiters.load(Ordering::Acquire) {
        return 0;
    }
    // 取出并清空本轮登记的 Waker
    let (wakers, len) = self.inner.lock().take_wakers();
    self.has_waiters.store(false, Ordering::Release);
    // 真正唤醒时已经不再持有内部锁
    for waker in wakers.iter().take(len) {
        unsafe { waker.assume_init_read() }.wake();
    }
    len
}
```

代码 3-5　PollSet 取出并唤醒等待者

`take_wakers` 在取出 Waker 的同时清空集合，因此后续事件不会再次使用这一批旧记录。

PollSet 只负责“有人需要重新检查了”，不会替业务对象判断条件是否真的满足。管道仍要检查缓冲区，子进程等待仍要检查状态，Futex 也要检查通知标记。

3.3.3 block_on 与通知竞争处理

Future 通过反复轮询判断操作是否完成，而同步系统调用需要在结果尚未产生时暂停当前任务。`block_on` 连接这两种执行方式：Future 返回 `Ready` 时，系统调用取得结果；返回 `Pending` 时，当前任务准备阻塞，把 CPU 交给其他任务。任务选择和现场切换仍由 3.2.3 的调度路径完成。

```
fn wake_by_ref(self: &Arc<Self>) {
    if let Some(task) = self.task.upgrade() {
        // 先记录通知已经到达
        *self.woke.lock() = true;
        // 再把阻塞任务放回合适的运行队列
        select_run_queue(&task).unblock_task(task, false);
    }
}

match future.poll(&mut context) {
    // 没收到通知，当前任务可以安全进入 Blocked
    Poll::Pending if !*woke => run_queue.blocked_resched(woke),
    // 通知先到，不再阻塞，只让其他任务先运行一次
    Poll::Pending => yield_now(),
    Poll::Ready(output) => return output,
}
```

代码 3-6　Waker 与 block_on 的阻塞唤醒配合

Future 返回 `Pending` 与通知到达可能发生在相邻时刻。Waker 先把 `woke` 设为真，再尝试唤醒任务；阻塞路径则在改变任务状态前检查这个标记。通知先到时，任务不再进入 `Blocked`，只让出一次 CPU 后重新轮询。阻塞先完成时，`unblock_task` 再把任务放回就绪队列。`woke` 记录“本轮轮询后是否已经收到通知”，补上了 Future 状态与任务状态之间的空隙。

图 3-5 对比了这两条路径的时序差异。

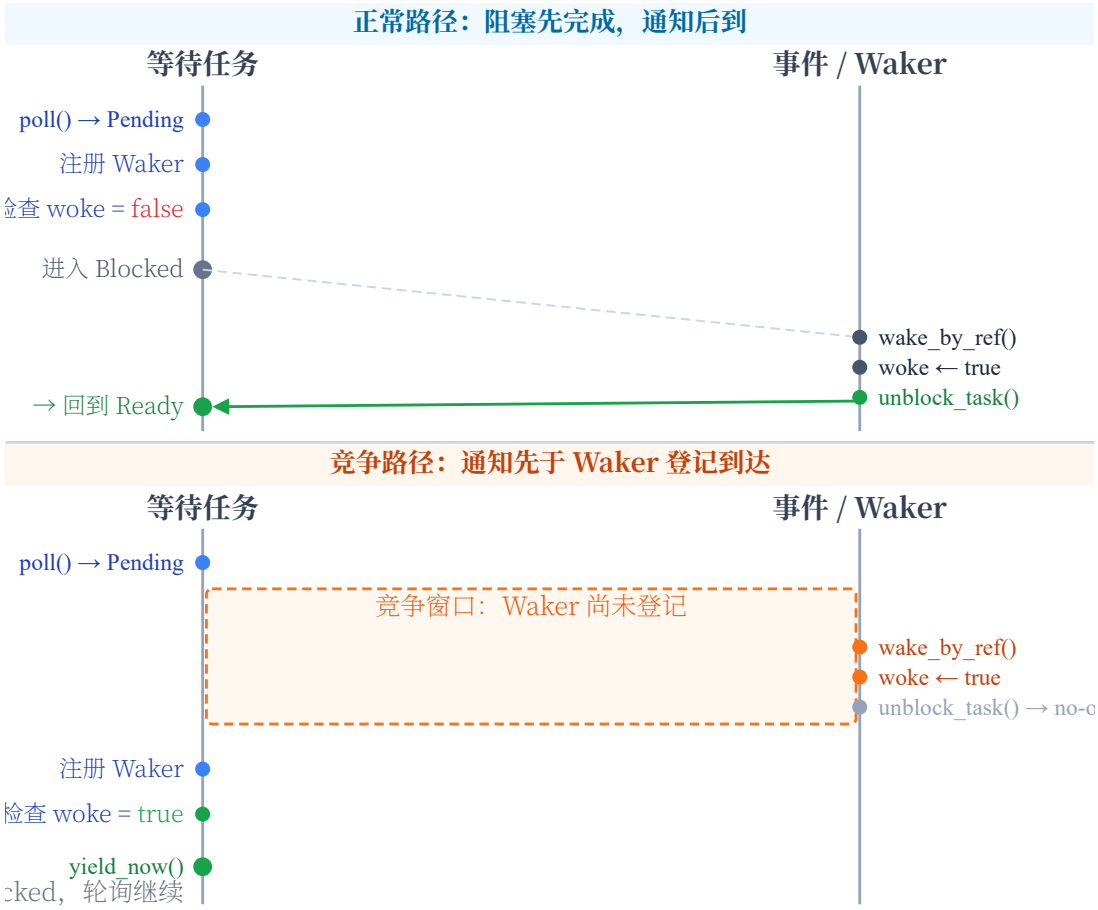


图 3-5　block_on 正常阻塞路径与 woke 标记覆盖竞争路径的时序对比

正常事件之外，信号也可以结束等待。`interruptible` 同时检查原来的 Future 和任务的信号状态。收到未屏蔽信号后，等待提前结束，上层系统调用再决定返回 `EINTR`、保存剩余时间或按信号规则重启。

等待因超时或信号提前结束后，还必须删除本轮登记。否则后续事件仍可能找到旧 Waker，既产生无效唤醒，也会干扰唤醒数量统计。PollSet 会清空已经取出的 Waker，Futex 则按等待项精确清理，具体过程将在 3.5 节说明。

3.4　通用阻塞机制的应用

3.3 节建立了”检查条件、登记 Waker、阻塞、收到通知后复查”的通用流程。本节以定时等待为例说明这套流程如何落地——业务条件是”截止时间是否到达”，任务恢复后仍要重新检查，不能把一次唤醒直接当作最终结果。子进程等待与信号机制紧密相关，通知路径在第六章说明。

3.4.1　定时等待与截止时间

定时等待先根据系统调用参数选择时间来源，再把等待目标统一换算为截止时间。单调时钟只向前推进，适合表示“再等待多久”；实时时钟对应系统日历时间，适合指定一个实际时刻。

时间参数分为相对时长和绝对时间。相对时长需要加到当前时间上，绝对时间本身就是截止时间；如果截止时间已经到达，等待立即结束。统一转换后，后续逻辑只需比较“当前时间是否达到截止时间”。同样的区分也用于修正后文 `FUTEX_WAIT_BITSET` 的超时语义。

`nanosleep` 使用单调时钟完成相对休眠。`clock_nanosleep` 还允许选择时钟，并通过 `TIMER_ABSTIME` 表示绝对时间。休眠期间如果收到未屏蔽信号，MossOS 会结束本次等待，向用户返回 `EINTR`，并写回尚未用完的时间。

3.4.2 定时器队列与到期处理

StarryOS 的定时事件可以按截止时间触发，但定时器重新设置后，旧事件可能仍留在队列中。仅凭截止时间无法判断取出的条目是否仍然有效。MossOS 改用 `BTreeMap` 保存定时器，并用截止时间 `deadline` 和递增序号 `seq` 组成唯一的 `TimerKey`。后台定时任务每次取出最早到期的条目，到期后更新定时器状态并发送相应信号。

```
struct TimerKey {
    // 定时事件应当触发的时刻
    deadline: Duration,
    // 区分截止时间相同的多个事件
    seq: u64,
}

struct AlarmRuntime {
    // 为新事件生成不重复的序号
    next_seq: u64,
    // 按到期顺序保存全部定时事件
    timers: BTreeMap<TimerKey, Entry>,
}
```

代码 3-7　定时器队列的关键数据结构

同一时刻可能有多个定时器到期，`seq` 使每个事件都有独立的键。重新设置或取消定时器时，内核删除旧键；后台任务取出事件后，还要确认它与定时器当前保存的键一致。只有匹配的事件才会触发信号，重新设置前留下的旧条目会被忽略。新增更早的定时器时，系统还会通知后台任务重新计算下一次等待时间。

每个线程还有一个时间管理对象 `TimeManager`，用来分别累计用户态和内核态运行时间。实际时间定时器 `ITIMER_REAL` 按经过时间发送 `SIGALRM`；用户时间定时器 `ITIMER_VIRTUAL` 只统计用户态时间；性能分析定时器 `ITIMER_PROF` 同时统计用户态与内核态时间。周期定时器到期后重新装载，一次性定时器到期后停止。

3.5 Futex 同步机制

3.5.1 基本思想与数据结构

Futex 把无竞争的同步留在用户态。线程先通过原子变量尝试获得锁，只有发现资源被占用时才进入内核等待。因此，大多数没有竞争的加锁和解锁不需要系统调用。

进入内核后，MossOS 使用 `FutexKey` 定位等待队列。进程私有 Futex 以用户地址为键；共享匿名映射和文件映射还要结合共享后端与区域偏移，保证不同进程即使把同一页映射到不同虚拟地址，也能找到同一个 Futex。

每个 Futex 表项保存等待队列和 `owner_dead` 状态。为了分清正常唤醒、超时和信号中断，MossOS 将单个等待项组织为：

```
pub struct WaitQueue {
    // 每个等待项依次保存：
    // 通知状态、唤醒位掩码和任务唤醒入口
    queue: SpinNoIrq<
        VecDeque<(Arc<AtomicBool>, u32, Arc<PollSet>)>
```

```
    >,
}
```

代码 3-8　Futex 等待队列的数据结构

元组中的 `AtomicBool` 是 `notified`，记录等待项是否已经被选中唤醒；`u32` 保存 bitset；`PollSet` 保存任务的唤醒入口。业务状态与唤醒动作由此分开：`notified` 防止通知丢失或重复处理，PollSet 负责让任务重新参与调度。

图 3-6 展示了 FutexKey 如何定位 WaitQueue，以及每个等待项三个字段的分工——`notified` 记录通知结果，bitset 过滤唤醒目标，PollSet 持有唤醒入口。

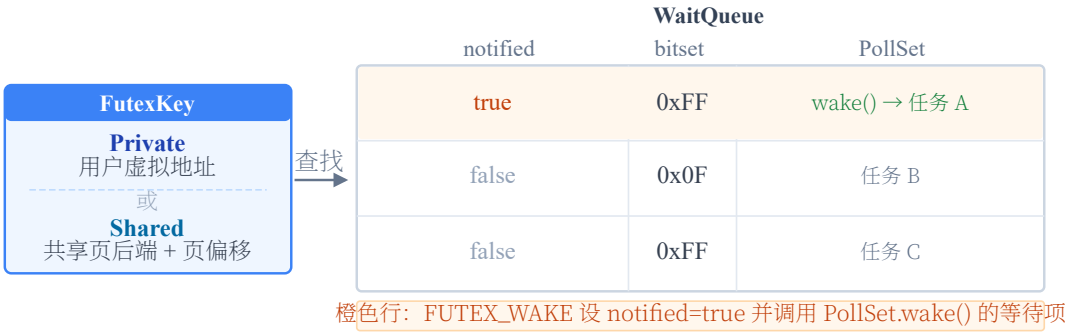


图 3-6　FutexKey、WaitQueue 与等待项三元组的结构关系

3.5.2　WAIT 与 WAKE

等待者只有在用户地址仍等于期望值时才应睡眠。系统调用入口先做一次快速检查，不相等就返回 `EAGAIN`；准备入队时，内核在持有等待队列锁的情况下再次读取用户值。第二次检查把用户值判断与入队操作放进同一个受保护区间，条件已经改变时，本次调用不再阻塞。

条件仍成立时，内核创建等待项并进入可中断、可超时的 Future。`FUTEX_WAKE` 从队列中选择指定数量的等待者，先把 `notified` 设为真，再调用 PollSet；被正常唤醒的等待项同时从队列删除，不会继续占用下一次 wake 的名额。

与 3.3.3 不同，这里额外处理 Futex 等待项入队后的竞争：`notified` 由 `FUTEX_WAKE` 写入，并在登记 PollSet 前后复查。它记录的是单个 Futex 等待项的通知结果，后续位掩码唤醒、超时清理和重新排队都会保留或处理该状态。

3.5.3　位掩码唤醒与重新排队

位掩码唤醒与重新排队也使用新增的 `notified` 状态和退出清理。`FUTEX_WAIT_BITSET` 为等待者附带位掩码，`FUTEX_WAKE_BITSET` 只唤醒 bitset 与目标 mask 有交集的任务，从而避免叫醒无关线程。

`FUTEX_REQUEUE` 先唤醒一部分任务，再把其余等待项移动到另一个 Futex 队列。`FUTEX_CMP_REQUEUE` 在此之前还会比较原地址的用户值，不匹配时返回 `EAGAIN`。移动过程中保留 `notified`、bitset 和 PollSet，不重新登记 Waker。

这类操作常用于 pthread 条件变量。广播时不必把所有线程同时叫醒争抢互斥量，可以先唤醒少量线程，再把其他等待者转移到互斥量对应的队列。如 图 3-7 所示，操作结束后原队列只剩余未被处理的等待者，其余项带着原有的 `notified`、bitset 和 PollSet 整体迁移到目标 Futex 队列。

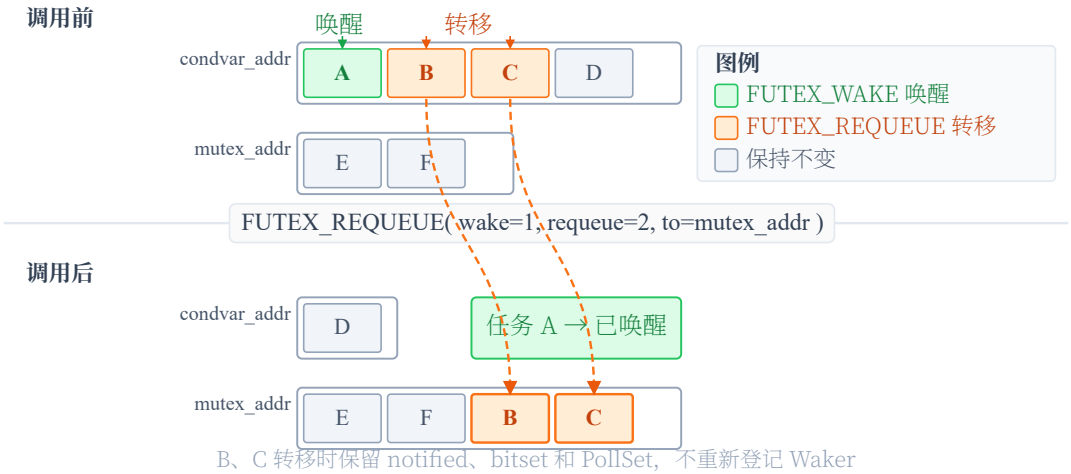


图 3-7　FUTEX_REQUEUE 将部分等待者唤醒、其余等待者转移至另一 Futex 队列

3.5.4　超时、中断与等待修正

Futex 等待可能正常收到通知，也可能先超时或被信号打断。MossOS 用 `interruptible(timeout(...))` 组合这三条路径。超时返回 `ETIMEDOUT`，信号中断返回 `EINTR`；两种提前结束都会按照 `notified` 的对象地址精确删除当前等待项。

原有实现没有把 `FUTEX_WAIT` 和 `FUTEX_WAIT_BITSET` 的时间语义分开。MossOS 按 Linux 规则处理：前者接收相对时长，后者接收绝对截止时间，并根据 `FUTEX_CLOCK_REALTIME` 选择实时时钟或单调时钟。

独立通知状态解决了“通知是否已经到达”的判断问题，前后复查覆盖登记窗口，退出清理则删除已经失效的等待项。三者分别处理通知丢失、重复处理和队列残留。相关修改修复了 LoongArch64 上 `futex_wait`、pthread cancellation 和部分休眠测试卡住的问题。

3.5.5　robust list 与线程退出清理

线程持有互斥量时如果直接退出，其他线程可能永远等不到解锁，从而造成死锁。针对这种情况，MossOS 完善了退出时的 Futex 清理。用户态线程库通过 `set_robust_list` 把 robust list 头指针交给内核，链表记录该线程可能持有的 Futex。

线程退出时，MossOS 按 `futex_offset` 找到对应 Futex，将 `owner_dead` 设为真，并唤醒一个等待者。被唤醒线程收到 `EOWNERDEAD` 后，由用户态线程库检查共享数据并恢复锁的一致性。链表遍历受 `ROBUST_LIST_LIMIT` 限制，损坏的用户链表不会让内核无限循环。

线程退出还会清零 `clear_child_tid`，并唤醒这个地址对应的 Futex。这条路径支持 `pthread_join` 等线程回收操作。

3.6　本章小结

StarryOS 已经具备 RR 调度、任务状态转换、上下文切换和基础事件等待，MossOS 没有另建一套调度器。本章的修改集中在调度器之外、重新进入调度器之前的那段路径：等待者登记状态，事件到来后留下通知，任务恢复时复查条件，等待提前结束时清理旧记录。

PollSet 缩短了持锁唤醒路径，`block_on` 用 `woke` 协调通知与阻塞，定时器用 `TimerKey` 识别仍然有效的事件，Futex 则通过 `notified`、双重检查和精确删除处理竞争。它们最终回到同一条流程：条件不满足时阻塞，条件可能变化时唤醒，复查成功后转为就绪，再由调度器安排运行。

第 4 章 内存管理

4.1 概述

StarryOS 已经具备用户虚拟内存管理的主体框架，其核心结构如 代码 4-1 所示。`AddrSpace` 记录可用地址范围和已装入的页表快照；`MemoryArea` 按起始地址组织互不重叠的虚拟区域，每个区域绑定一个 `Backend` 决定页面从何处取得；`Backend` 封装四种页面来源。多级页表在缺页时按需装入物理页，`fork` 通过 `AddrSpace::try_clone` 复制区域描述并对私有可写页面执行 COW 保护。

```
AddrSpace
├── va_range              可用虚拟地址上下界
├── areas: MemorySet
│   └── BTreeMap<VirtAddr, MemoryArea>
│       ├── va_range      连续的虚拟地址范围
│       ├── flags         读 / 写 / 执行 / 用户态权限
│       └── backend       Linear | Cow | Shared | File
└── pt: PageTable         已装入的虚拟页 → 物理页快照
```

代码 4-1　AddrSpace 三层结构概览

MossOS 沿用这套框架，我们的工作集中在 `mmap` 系统调用的语义补全上。StarryOS 的 `sys_mmap`（`kernel/src/syscall/mm/mmap.rs`）主干已能处理 `MAP_PRIVATE`、`MAP_SHARED` 和 `MAP_ANONYMOUS` 的基本组合，对应关系如 表 4-1 所示。但若干标志和配套接口仍为占位实现——`sys_mlock` 直接返回成功而未执行实际锁定，`sys_madvise` 忽略 advice 参数，`sys_mremap` 强制以私有匿名映射重建目标区域。MossOS 将这些占位实现补全为完整语义，将 mmap 相关代码从 337 行扩展到 1023 行。

映射请求	**Backend**	行为
`MAP_PRIVATE` `MAP_ANONYMOUS`	Cow	建立私有匿名区域，首次访问时清零分配
`MAP_PRIVATE` 文件映射	Cow	从文件读取初始内容，写入后仅修改私有副本
`MAP_SHARED` `MAP_ANONYMOUS`	Shared	分配物理页槽位，多进程共享同一组页面
`MAP_SHARED` 文件映射	File	通过文件缓存访问，多映射看到同一份内容
设备物理范围	Linear	按固定偏移直接映射设备提供的物理地址

表 4-1　mmap 标志与 Backend 的对应关系（StarryOS 基线已具备）

表 4-2 总结了 MossOS 在 mmap 上的各项补全及其动机。

机制	基线状态	MossOS 的处理
栈自动增长	`MAP_GROWSDOWN` 已定义但忽略	缺页时自动向下扩展一页，避免大局部变量或深层递归直接触发 SIGSEGV
mremap 迁移	不处理范围重叠	支持 `MREMAP_FIXED`，先解除重叠区域再建立新映射，防止数据丢失
WIPEONFORK	不存在	fork 后在子进程中清零标记区域，避免父子进程共享 malloc 元数据导致堆损坏
/proc/[pid]/maps	不存在	输出 Linux 兼容的映射列表，供 gdb、pmap、ASAN 等工具获取进程内存布局
PROT_WRITE	W=1,R=0 导致页表映射失败	自动补 READ 位但保持用户可见 prot 不变，规避 RISC-V 特权规范的可写页必须可读的限制
SIGBUS 追踪	无	文件尾外写入触发 SIGBUS 而非 SIGSEGV，区分“地址合法但存储不可用”与“地址非法”

表 4-2　MossOS 在 mmap 上的语义补全

下面逐项展开这些机制。

4.2 栈自动增长

用户栈在程序启动时通常只分配几页。C 程序中声明大型局部数组、深层递归调用或 alloca 动态分配都可能超出初始栈空间。如果内核在栈耗尽时直接发送 `SIGSEGV`，大量合法程序将无法运行。Linux 的做法是：栈区域以 `MAP_GROWSDOWN` 标记，缺页时自动向下扩展一页，而非拒绝访问。

代码 4-2 展示了 GROWSDOWN 缺页的处理路径。内核先调用 `AddrSpace::handle_page_fault` 尝试按已注册区域填页；若未命中任何区域，则在返回前调用 `try_growdown_fault` 检查是否有 GROWSDOWN 区域可以扩展。全局 `GROW_DOWN_AREAS` 表以进程 pid 为键，记录每个区域的当前边界。若缺页地址落在某个区域起始地址以下的 4KiB 范围内，则进入 `block_growdown`——为新页面建立物理映射并向下扩展区域边界。

```
handle_page_fault(vaddr)
├── areas.find(vaddr) 命中 → 正常 Backend::populate
└── 未命中
    └── try_growdown_fault(vaddr)
        ├── 遍历 GROW_DOWN_AREAS[pid]
        ├── 地址在 [area.low, area.start) 且距 start ≤ 4KiB?
        │   ├── 是 → block_growdown(area)
        │   │       ├── 分配清零页 + 建立页表映射
        │   │       └── 更新 area.start -= PAGE_SIZE
        │   └── 地址触及 area.low → SIGSEGV
        └── 无匹配 → SIGSEGV
```

代码 4-2　GROWSDOWN 缺页处理流程

扩展不是无限制的。每个 GROWSDOWN 区域有一个预设的 `low` 下界（通常为栈底预留的保护页），地址触及下界后不再扩展，转为 `SIGSEGV`。这防止了栈无限增长侵入堆或其他映射区域。扩展后的新页面由 Cow Backend 管理，首次访问时清零分配，`fork` 时通过 COW 与父进程共享。

4.3 mremap 迁移

`mremap` 在原始区域后方空间不足以容纳新大小时，需要将映射迁移到新地址。StarryOS 基线的做法是以 `MAP_PRIVATE | MAP_ANONYMOUS` 在新地址重建映射、复制数据、再解除旧映射。这能完成数据搬迁，但引入了两个实际问题。

第一个问题是新旧地址范围可能部分重叠。如果新地址落在旧地址范围中间，朴素的“先建新映射再删旧映射”会导致新建的区域被后续的 unmap 操作误删。MossOS 改为在建立新映射前先检查是否与旧范围重叠，若重叠则先解除旧映射的重叠部分，再在新地址建立映射。

第二个问题是迁移后原始区域的 Backend 类型信息丢失——文件映射退化为匿名映射，`/proc/[pid]/maps` 不再能显示正确的路径和偏移。MossOS 在迁移后保留原始区域在 `MMAP_RECORDS` 表中的记录，确保 `maps` 输出仍能反映正确的权限和映射类型。迁移路径仍以私有匿名映射重建（MossOS 未改变这一保守策略），但 `MREMAP_MAYMOVE` 未设置且后方空间不足时，正确返回 ENOMEM 而非强行迁移。

4.4 MADV_WIPEONFORK

多线程程序中，`fork` 只复制调用线程，子进程的地址空间是父进程的完整快照。如果父进程在 fork 前通过 `malloc` 分配了内存，子进程继承的堆中包含了父进程 allocator 的内部元数据（如 jemalloc 的 extent 链表）。此时若子进程尝试 `free` 这些继承而来的指针，会操作与父进程共享的元数据结构，导致堆损坏。

`madvise(MADV_WIPEONFORK)` 允许 allocator 在分配关键数据结构时标记对应内存，指示内核在 `fork` 后于子进程中将该区域清零。这样子进程拿到的是干净的匿名页，不再看到父进程的 allocator 元数据。

MossOS 用全局 `WIPEONFORK_AREAS` 表以 `(pid, addr_range)` 为键记录标记区域。`fork` 时 `inherit_wipeonfork_ranges` 将父进程标记复制给子进程，子进程遍历区域逐页分配清零页——对文件映射替换原缓存页，对匿名映射替换 COW 共享的父进程页。`MADV_KEEPONFORK` 从表中移除标记，后续 fork 不再清空该区域。

4.5 /proc/[pid]/maps

调试器（gdb、lldb）、内存分析工具（ASAN、valgrind）和 `pmap` 命令都需要读取 `/proc/[pid]/maps` 来获取进程的内存布局——哪些地址范围被映射、权限是什么、是私有还是共享映射、对应的文件路径。没有这个接口，这些工具无法定位代码段、堆和栈的位置，调试功能基本不可用。

MossOS 的 `mmap_proc_maps()` 取进程地址空间锁后遍历 `MemoryArea`，对每个区域生成一行文本：起始-结束地址、`r/w/x/p` 或 `r/w/x/s` 权限字符串（`p` 表示私有映射，`s` 表示共享映射）、偏移和占位符字段。权限判断依赖 MossOS 在 `MMAP_RECORDS` 表中为每次 `mmap` 存储的 `shared` 属性和 `prot` 标志。`mprotect` 修改权限时同步更新该表中的记录，确保 `maps` 输出与实际权限一致。

4.6 PROT_WRITE 与 SIGBUS

RISC-V 特权架构规范（§4.3.2）要求：当页表项的 W 位为 1 时，R 位也必须为 1。换言之，硬件不允许“只写”页面。用户以 `PROT_WRITE`（不含 `PROT_READ`）调用 `mmap` 时，直接设置 W=1、R=0

会导致 pt.map() 在 TLB 刷新阶段触发硬件异常，整个 mmap 调用失败。受影响的典型场景包括某些 JIT 编译器分配的 write-only 代码缓冲区。

MossOS 的修复是在 MmapProt 到 MappingFlags 的转换中做一次补偿：若 WRITE 已设置而 READ 未设置，自动补上 READ 位用于硬件页表，但用户可见的 MmapProt 保持不变——调用者后续通过 mprotect 或 /proc/maps 查询权限时，仍得到 PROT_WRITE，而非 PROT_READ | PROT_WRITE。

文件映射还有一个类似的语义差异：访问超出文件尾的映射地址时，地址在 MemoryArea 范围内是合法的，但后端文件无法提供对应数据。这种情况应触发 SIGBUS（总线错误，表示“地址合法但存储不可用”），而非 SIGSEGV（段错误，表示“地址根本不在任何映射区域内”）。MossOS 用全局 SIGBUS_AREAS 表记录文件映射中超出文件有效范围的区域，缺页处理在发现文件页偏移超出文件长度时先查表确认，若是则触发 SIGBUS。进程退出时 release_process_mmap_resources 统一清理该进程的 GROWSDOWN、SIGBUS 和 WIPEONFORK 记录。

4.7 COW 与缺页路径的改进

StarryOS 的 COW 机制已通过物理页引用计数实现。MossOS 在此基础上做了两项改进。

引用计数类型从 u8 扩展为 usize。u8 的 255 上限在单进程内足够——同一进程内很少有超过 255 个区域共享同一物理页。但如果进程链式 fork（shell 脚本每行命令 fork 一次），多层父子进程可能累积共享同一物理页，255 的理论上限有被触及的风险。usize 消除了这个边界。

Cow Backend 增加 mapped_pages: AtomicUsize 计数器，统计当前已装入页表的页面数量。懒加载区域如果从未触发缺页，父页表中就没有需要 COW 保护的物理页。fork 时通过原子读取检查该字段，若为零则跳过整个区域的页表遍历和只读保护设置，直接复制区域描述即完成子地址空间的构建。频繁 fork/exec 场景（如 shell 脚本循环）中，多数区域在 fork 发生时尚未被访问过，跳过无效页表遍历的收益显著。

在缺页路径上，文件映射 Backend 增加了轻量预读探测。用 last_fault_page 记录上一次缺页的页号：单页读缺页时若当前页号恰好比上次大 1，判定为顺序扫描，将后续 4 页预热入页缓存。预热不建立页表映射，后续页仍触发缺页来建立映射，但数据已在缓存中，缺页等待时间更短。随机访问时 last_fault_page 不连续，预读不触发，单次 fault 延迟不受影响。

Shared Backend 的页面分配也改为惰性策略。StarryOS 的 SharedPages 在创建时立即为全部槽位分配物理页——对于上百页的共享映射，即使只实际访问其中几页，也需要承担全部页面的分配开销和 fork 后的引用管理。MossOS 改为 Option<PhysAddr> 槽位，物理页在首次 get_or_alloc 时才分配，fork 后子进程共享同一组 SharedPages 对象，未访问的槽位不占用物理内存。

4.8 本章小结

MossOS 的内存管理增量集中在 mmap 系统调用的语义补全。栈自动增长解决了用户栈动态扩展的需求，避免了大型局部变量和递归场景下的误杀；mremap 重叠处理和 WIPEONFORK 分别消除了数据搬迁中的数据丢失风险和 fork 后 allocator 元数据共享导致的堆损坏；/proc/[pid]/maps 为调试器和内存分析工具提供了基本的进程内存可见性；PROT_WRITE 修复和 SIGBUS 追踪补全了 RISC-V 特有的硬件限制和文件映射的边界语义。COW 引用计数扩展和 mapped_pages 跳过优化减少 fork 密集场景的额外开销，缺页预读和 Shared 懒分配分别为顺序访问和共享映射带来了延迟与内存的改善。

第 5 章 文件系统

5.1 概述

从 Unix 的设计哲学 —— “Everything is a file.”出发，文件系统要解决的核心问题是：让用户程序用同一套接口去访问性质完全不同的对象——磁盘上的 ext4 文件、内存里的临时文件、/proc 下的内核信息、/dev 下的设备节点。

StarryOS 为这个问题提供了一套完整的基础框架。分四层：axfs-ng-vfs 定义文件系统、目录节点、文件节点的接口；axfs 实现路径解析、页缓存和 ext4/FAT32 后端；kernel/src/file/ 将 VFS 对象统一进文件描述符表；syscall/fs/ 把系统调用参数翻译为上层调用。

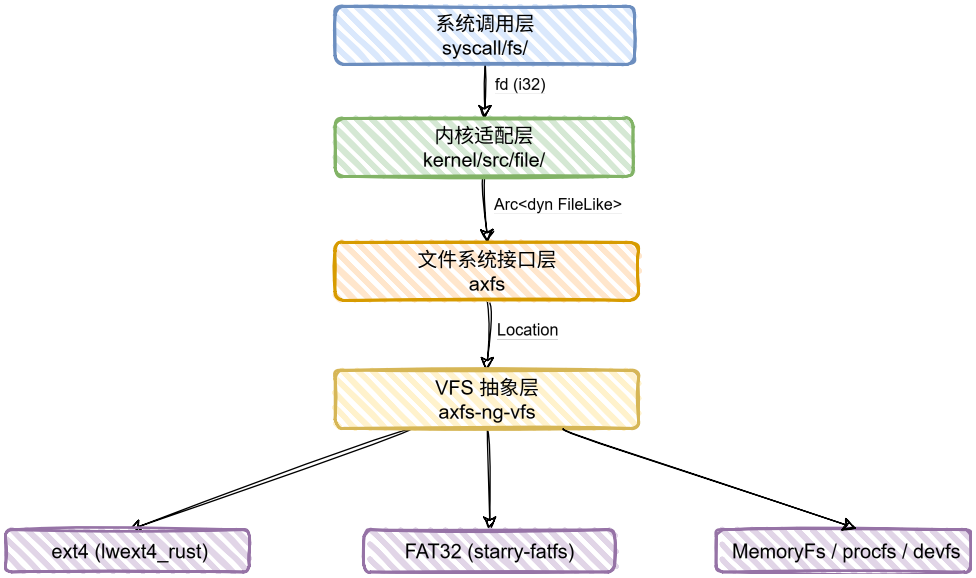


图 5-1　文件系统分层架构

在这套框架中，StarryOS 已经提供了几个关键设计：

- **Location**：将目录项与所属挂载点绑定为统一句柄，路径解析穿越挂载点时自动切换文件系统；
- **Mountpoint** 树：子节点用 `Weak` 引用，卸载时自动释放，跨设备检测用 `Arc::ptr_eq` 对比挂载点指针；
- **NodeFlags**：四个标志位（`STREAM`、`NON_CACHEABLE`、`ALWAYS_CACHE`、`BLOCKING`）让节点声明自身 I/O 契约，上层无需判断节点类型；
- **FsContext** + 路径解析：每个任务独立的 `root_dir` 和 `current_dir`，四个解析变体覆盖不同 syscall 语义，符号链接循环检测上限 40 层；
- 文件系统后端：lwext4_rust 接入 ext4，starry-fatfs 接入 FAT32，MemoryFs 提供内存文件系统，SimpleFs 框架支撑 procfs/devfs。

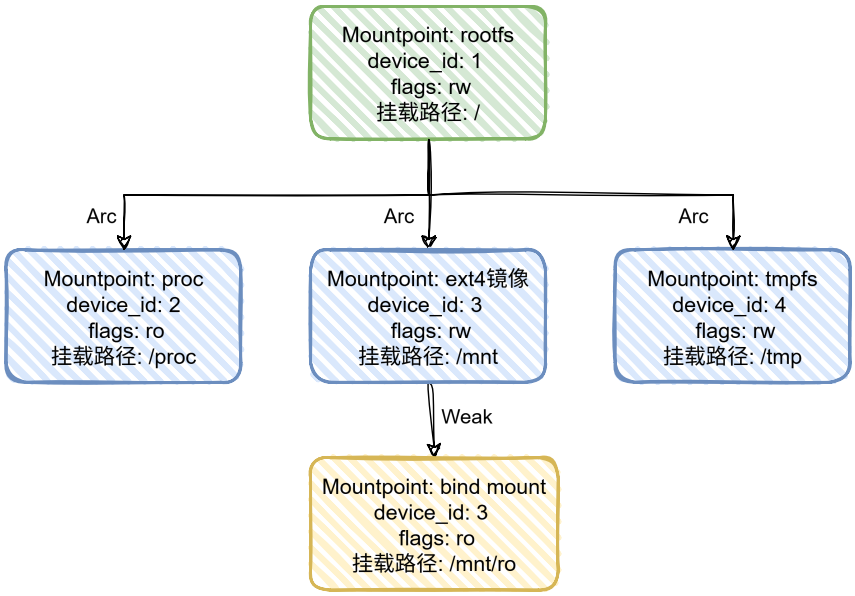


图 5-2　Mountpoint 树

MossOS 在这套框架上的工作集中在六个方面：挂载系统调用的完善、新式挂载 API 的实现、inode 元数据缓存的引入、扩展属性的支持、文件事件通知（inotify 与 fanotify），以及 I/O 路径的性能优化。下面按这几个方向展开。

5.2　挂载子系统

挂载解决的核心问题是：把独立的文件系统拼到同一棵目录树下。`mount` 调用在 `Mountpoint` 树上添加节点，路径解析穿越目录时自动切换到对应文件系统的根——这个设计来自 StarryOS。StarryOS 的 `sys_mount` 当前支持 `tmpfs`，flags 参数尚未处理；`sys_umount2` 同样未包含权限和状态检查。MossOS 在此基础上扩展了挂载操作的类型和语义。

5.2.1　传统挂载接口的完善

`sys_mount` 在 MossOS 中除了普通挂载外，支持了三种改变已有挂载关系的操作模式：

标志	含义
`MS_BIND`	bind mount：让已有目录树在另一条路径下可见，直接复用源挂载点对象，不创建新文件系统实例
`MS_MOVE`	迁移挂载点：将挂载从一个路径移到另一个路径，底层做 bind + unmount 原子操作
`MS_REMOUNT`	修改现有挂载的标志（如读写→只读），修改前检查是否有打开的写文件

表 5-1　sys_mount 新增的操作模式

此外，MossOS 增加了权限检查（要求 euid == 0），支持 10 种挂载标志（`MS_RDONLY`、`MS_NODEV`、`MS_NOEXEC`、`MS_NOSUID`、`MS_NOATIME`、`MS_NODIRATIME`、`MS_RELATIME`、`MS_STRICTATIME`、`MS_REMOUNT`、`MS_NOSYMFOLLOW`），并将 `fs_type` 从单一的 `"tmpfs"` 扩展到 ext4、FAT32 及 LTP 兼容的多种文件系统。

`sys_umount2` 同样做了大量补全：

标志 / 行为	说明
MNT_DETACH	延迟卸载：即使文件被占用也移除挂载，busy 文件在最后一个 fd 关闭后自动释放
MNT_EXPIRE	两阶段过期：首次调用标记到期，再次调用时若无打开文件则卸载，否则返回 EAGAIN
UMOUNT_NOFOLLOW	不跟随符号链接，直接操作路径本身
卸载前清理	移除页缓存（drop_file_caches_by_device）、刷新文件系统脏数据（flush）、发送 inotify IN_UNMOUNT 事件
忙碌检查	MNT_DETACH 未设置时检查是否有打开的写文件，有则返回 EBUSY

表 5-2　sys_umount2 新增功能

5.2.2 新式挂载 API

传统 sys_mount 有一个历史包袱：调用者必须同时拥有源设备访问权限和目标路径挂载权限。容器场景里这很别扭——容器进程应该能在自己的命名空间里自由挂载，却因为没有宿主机路径权限而卡住。Linux 5.2 引入的新式挂载 API 将配置和挂载拆为两步，StarryOS 没有实现这套接口，MossOS 补上了主要路径。

```
fd = fsopen("ext4")                          // 创建文件系统上下文
fsconfig(fd, "source", "/dev/loop0")  // 配置设备路径
fsconfig(fd, "cmd", "create")           // 完成配置，准备挂载
mfd = fsmount(fd, 0, 0)                        // 实例化为挂载对象 fd
move_mount(mfd, "", "/mnt", MOVE_MOUNT_F_EMPTY_PATH)
```

代码 5-1　新式挂载 API 的完整调用流程

五个新系统调用各有明确的职责：

- fsopen：按文件系统类型名创建上下文，返回 FsContextFd；
- fsconfig：对上下文进行配置（设置 source 路径、发出 create 命令），支持 FSCONFIG_SET_STRING 和 FSCONFIG_CMD_CREATE 两种操作；
- fsmount：将配置完成的上下文实例化为 DetachedMountFd，此时文件系统已创建但尚未附加到任何路径——“分离的挂载”；
- open_tree：打开已挂载的文件系统树作为 detached mount fd（OPEN_TREE_CLONE 模式），为 bind mount 提供基于 fd 的接口；
- move_mount：将 detached mount fd 挂载到目标路径，支持 MOVE_MOUNT_F_EMPTY_PATH 标志（空路径表示直接挂载 fd 本身）。

DetachedMountFd 是一个关键设计——它在挂载操作期间作为“尚未附加的挂载对象”存在，可以是全新文件系统（Filesystem { fs, mount_flags }）或已有挂载树的引用（Bind { source: Location }）。这个中间态让配置和挂载的分离成为可能。

5.2.3 Loop 设备

LTP 文件系统测试中大量用例需要在运行时把镜像文件（如 ext4.img）当块设备挂载：先 mkfs.ext4 格式化一个普通文件，再 mount 它。要支持这个场景，需要能把普通文件当成块设备用。

MossOS 为此实现了 LoopFileBlockDevice：

```rust
pub struct LoopFileBlockDevice {
    file: FileBackend,    // 被当作块设备使用的普通文件
    blocks: u64,          // 文件总块数 = file.len() / block_size
}

impl BlockDriverOps for LoopFileBlockDevice {
    fn read_block(&mut self, block_id: u64, buf: &mut [u8]) -> DevResult {
        let offset = self.checked_range(block_id, buf.len())?;
        // 循环 read_at 直到读完，处理短读
    }
    fn write_block(&mut self, block_id: u64, buf: &[u8]) -> DevResult {
        // 块号 → 文件偏移的换算：offset = block_id * block_size
    }
    fn flush(&mut self) -> DevResult {
        self.file.sync(false)  // 将脏数据写回底层文件
    }
}
```

代码 5-2　LoopFileBlockDevice 将普通文件适配为块设备接口

核心逻辑是块号到文件偏移的换算：块设备第 N 块对应普通文件从 `N * block_size` 开始的一段内容。ext4 后端仍按块读写，`LoopFileBlockDevice` 把这些块访问翻译成对镜像文件的 `read_at/write_at`。`sys_mount` 在识别到 `/dev/loop*` 路径时，将对应文件包装后传给 ext4 后端，镜像文件挂载走普通 ext4 路径，不需要单独处理。读写循环处理了短读/短写（底层文件系统可能一次未返回全部请求字节），只读 loop 设备以非 `MS_RDONLY` 挂载时显式拒绝。

5.3　inode 元数据缓存

频繁的 `stat` 调用和读写伴随的时间戳更新如果每次都穿透到底层文件系统，会在 I/O 密集场景产生大量元数据写——进程只是想读文件，却要附带一次写的开销。StarryOS 的 `metadata()` 直接访问底层节点，未引入额外的缓存层。MossOS 在 VFS 层增加了一套 inode 元数据缓存，以减少频繁穿透底层文件系统带来的开销。

```rust
type InodeMetadataCacheKey = (usize, u64);          // (fs_ptr, inode)，唯一定位
static INODE_METADATA_CACHES:
    Lazy<Mutex<BTreeMap<InodeMetadataCacheKey, SharedInodeMetadataCache>>>
= ...;

struct InodeMetadataCache {
    metadata: Metadata,             // 缓存的 size/atime/mtime/ctime 等
    dirty_size: bool,               // 文件大小变更但未提交
    dirty_update: MetadataUpdate,   // 时间戳/权限等待提交的更新
}
```

代码 5-3　inode 元数据缓存：以 (fs_ptr, inode) 为键的全局 BTreeMap

缓存以 `(fs_ptr, inode)` 为键存入全局 `BTreeMap`，同一文件被多个进程、多个 fd、多条路径访问时共享同一份缓存。目录的 nlink 随子节点增删变化，缓存容易读到过期值，被排除在外；`NON_CACHEABLE` 节点（procfs 等）内容动态生成，同样不进入缓存。

元数据更新采用延迟提交策略：各字段的修改先合并到 `dirty_update` 中，`sync` 或文件关闭时统一提交，将多次独立的元数据写合并为一次底层调用。atime 遵循 relatime 策略——只有当 atime 早于 mtime 或 ctime、或距上次更新超过 24 小时时，读操作才触发 atime 更新。`MS_NOATIME` 和 `MS_STRICTATIME` 挂载标志可以覆盖这个策略。这个设计让大多数只读访问不再产生元数据写开销。

缓存在以下时机被同步或失效：`fsync/close` 时调用 `flush_metadata_cache` 将脏数据写回；文件被删除时 `invalidate_inode_metadata_cache` 先 flush 再移除；设备卸载时 `drop_file_caches_by_device` 批量清理。

5.4 扩展属性

扩展属性（xattr）是文件系统上以键值对形式存储的元数据，用于 ACL、SELinux 安全标签、文件校验和等场景。与 inode 自带的 mode/uid/gid 不同，xattr 由用户态自由定义和扩展。StarryOS 尚未包含 xattr 相关接口，MossOS 从 syscall、VFS trait 和 inode 存储三个层面补充了这套机制。

层次	MossOS 实现
syscall 层	12 个系统调用：setxattr/getxattr/listxattr/removexattr × 三个变体（路径版、l-符号链接版、fd 版），共 291 行
VFS trait 层	NodeOps 新增 4 个方法（set_xattr/get_xattr/list_xattr/remove_xattr），默认返回 OperationNotSupported；Location 通过 inherit_methods 透出对应方法
存储层	per-inode 的 XattrStore（`Mutex<BTreeMap<Vec<u8>, Vec<u8>>>`）挂在 Location 的 TypeMap 上，惰性创建；ext4 通过 lwext4 接口直接读写磁盘 xattr

表 5-3　xattr 三层实现

实现中处理了几个规范语义：

- 命名空间：支持 `user.`、`security.`、`trusted.`、`system.` 四个前缀，`user.` 仅允许普通文件和目录使用；
- 标志语义：`XATTR_CREATE` 要求属性不存在才写入，`XATTR_REPLACE` 要求存在才替换，与 `setxattr(2)` 完全一致；
- 大小限制：属性名最长 255 字节，值最长 65536 字节；
- **inode flags** 集成：immutable 或 append-only 文件自动拒绝 xattr 写入。

5.5 文件事件通知

用户态程序需要知道文件系统上发生了什么——备份工具感知文件修改，安全软件在文件访问时介入。Linux 提供 inotify 和 fanotify 两套机制覆盖不同场景。StarryOS 尚未包含这两套接口，MossOS 补充了主要路径。

5.5.1 inotify

inotify 面向具体文件或目录的变化监视。用户态对目标路径添加 watch，之后 create、delete、modify、attrib、move 等操作产生的事件进入事件队列，通过 `read` 取出。

实现上，一个 inotify fd 维护两张表：

```
struct Target { dev: u64, ino: u64, is_dir: bool }
struct Watch  { wd: c_int, target: Target, mask: u32 }
struct Event  { wd: c_int, mask: u32, cookie: u32, name: Option<String> }

pub struct InotifyFd {
    watches: Mutex<Vec<Watch>>,          // 当前 fd 上注册的所有 watch
    events: Mutex<VecDeque<Event>>,      // 等待用户态 read 的事件队列
    poll: PollSet,                       // 新事件到达时唤醒 read/poll
}
```

代码 5-4　inotify 的 watch 表与事件队列

事件投递发生在 VFS 层的创建、删除、重命名、属性更新路径上——这些路径在 StarryOS 中已经存在，MossOS 的工作是在这些路径上插入通知点。移动操作产生成对的 `IN_MOVED_FROM` 和 `IN_MOVED_TO` 事件，通过相同的 cookie 关联，让接收方能识别是同一次 rename。卸载时向所有相关 watch 发送 `IN_UNMOUNT`。

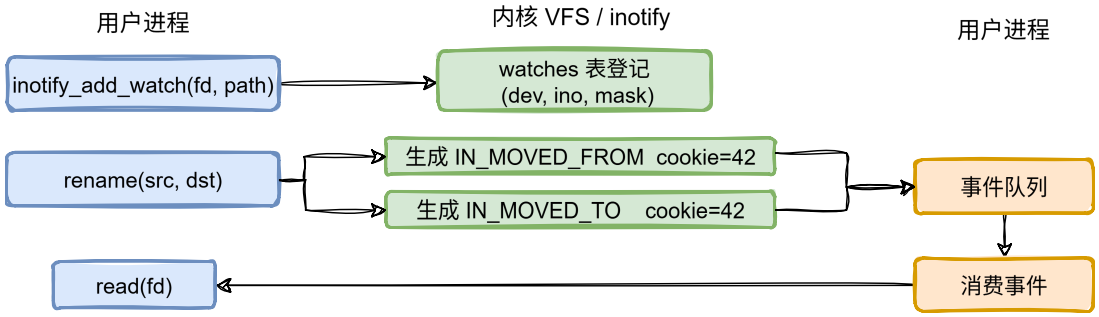


图 5-3　inotify 从注册 watch 到用户态取出事件的完整数据流

5.5.2 fanotify

fanotify 解决 inotify 覆盖不了的两个场景：监视范围可以是一整个挂载点甚至文件系统（而非单个路径），以及能在访问发生时主动介入（而非事后通知）。

mark 可以挂在单个文件、目录或整个挂载点上：

```
enum FanotifyMarkKind { Inode, Mount, Filesystem }

struct FanotifyMark {
    target: FanotifyTarget,
    kind: FanotifyMarkKind,     // mark 的作用范围
    mask: u64,                  // FAN_OPEN/FAN_ACCESS/FAN_CREATE 等
    ignored_mask: u64,          // 临时忽略的事件集合
}

struct FanotifyEvent {
    mask: u64,
    file: Option<Arc<dyn FileLike>>,  // 需要返回 fd 时保留文件对象
    fid: Option<FanotifyTarget>,      // FID 模式下报告的文件身份
    permission_id: Option<u64>,       // 权限事件等待用户态响应
}
```

```rust
pub struct FanotifyFd {
    marks: Mutex<Vec<FanotifyMark>>,
    events: Mutex<VecDeque<FanotifyEvent>>,
    permissions: Mutex<Vec<FanotifyPermission>>,  // 尚未 allow/deny 的权限事件
    poll: PollSet,
}
```

代码 5-5　fanotify 的 mark、事件队列与权限等待表

权限事件（FAN_OPEN_PERM、FAN_ACCESS_PERM）是 fanotify 最特殊的部分：内核在文件被打开或访问时先等待监听进程响应，再决定是否放行。触发进程在此期间阻塞，监听进程通过写入 allow 或 deny 响应。安全软件（防病毒、DLP）依赖这个机制在文件访问路径上做拦截。

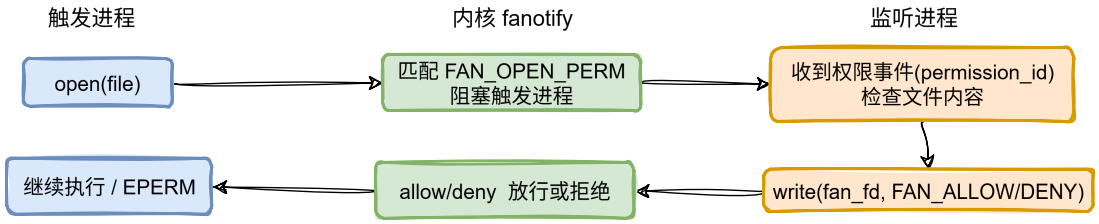


图 5-4　fanotify 权限事件的三方交互时序

5.5.3 无监听快速路径

inotify 和 fanotify 需要在 read、write、open、close 等热路径上插入事件投递逻辑。没有注册监听者时，这些逻辑不应带来任何额外开销。MossOS 用全局计数和掩码做早期过滤：inotify 维护全局 watch 计数，为零时通知函数立即返回；fanotify 维护全局兴趣掩码，对当前事件无命中时跳过后续所有处理。两处检查都只需一次原子读，无监听者时不进入任何路径解析、mark 匹配或事件队列操作。

5.6 I/O 性能优化

文件系统的性能差异在 StarryOS 和 MossOS 之间非常显著：iozone 测试中，顺序写提升约 7–13 倍，顺序读提升约 5 倍，随机读写提升约 5–9 倍。这些提升根源于页缓存和元数据缓存的几项关键优化。

5.6.1 页缓存增强

StarryOS 已经有基础的页缓存机制，但 MossOS 做了三个关键改进。

全局索引 + **per-file LRU**。StarryOS 的页缓存文件查找路径较长。MossOS 以 `(fs_ptr, inode)` 为键建立全局 `BTreeMap`，同一文件被多个 fd 打开时找到同一份缓存记录。每个文件的缓存数据放在独立的 LRU 中，默认容量 256 个 4KiB 页（1MiB），单个文件的大规模顺序扫描不会挤出其他文件的缓存。

避免 **read-before-write**。页缓存以 4KiB 为单位操作，朴素的实现在目标页缺失时会先读出完整旧页再部分覆盖——这对修改已有文件中间段是必要的，但对新文件顺序写或 EOF 之后 append 而言是浪费。MossOS 判断写入是否完全覆盖了页内旧有效区域，如果是则直接分配清零页而不调用底层 `read_at()`，消除了新页写入前的无效读取。

脏页合并写回。脏页按页号排序，连续页合并成一批写入，每批最多 16 页（64KiB）。相比逐页提交，顺序写场景下的块设备请求数量明显减少。

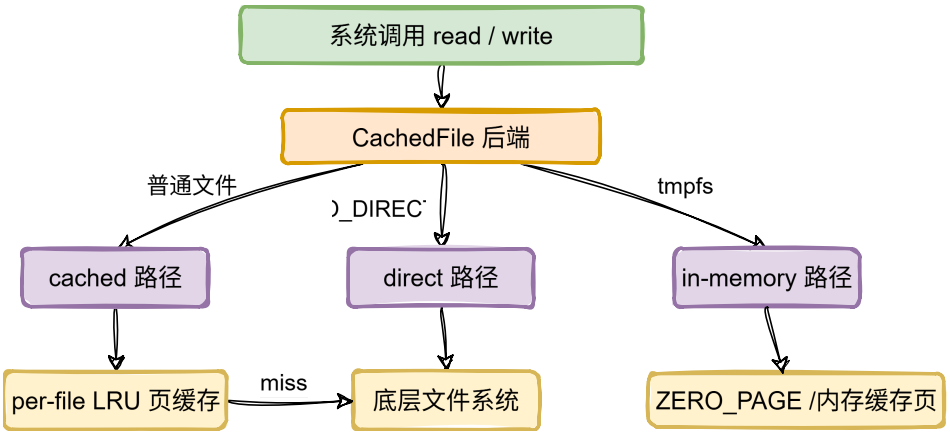


图 5-5　I/O 读写三条路径：cached、direct 和 in-memory

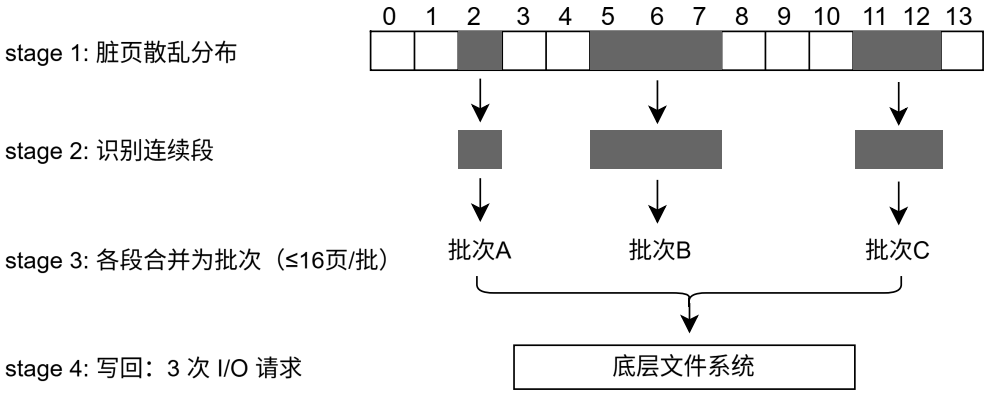


图 5-6　脏页按页号排序后合并为批次写回

Direct I/O 与 Cached I/O 之间的一致性也得到了处理：direct 读前先将目标范围内的 dirty cached 页写回，direct 写后使对应干净页失效，两条路径在重叠区域不会读到不一致的内容。

5.6.2　元数据缓存

元数据缓存的实现细节已在第三节中描述。这里补充其在性能上的意义：IOPS 密集场景下，大量 `stat/fstat` 调用不再穿透底层文件系统；atime 的 relatime 策略使绝大多数只读访问不产生任何元数据写；延迟提交将多次独立的元数据更新合并为一次底层调用。

5.6.3　预读

预读在应用程序实际访问页面之前提前将文件内容放入页缓存，降低后续访问的等待时间。StarryOS 的 `fadvise(WILLNEED)` 仅做参数检查，未执行实际预读，MossOS 补全了这条路径：`sys_readahead()` 和 `fadvise(POSIX_FADV_WILLNEED)` 找到文件对应的 cached backend，按请求范围同步地将页面放入缓存，单次最多 1MiB。direct I/O 场景保持 no-op，符合绕过缓存的语义。

mmap 访问时每次 page fault 只处理当前缺失页。MossOS 在 fault 后检查 `last_fault_page`：若当前页号比上次大 1，说明是顺序扫描，额外将后续 4 页预热入缓存。预读只放入页缓存不建立页表映射，后续 fault 命中缓存时延迟更低，且不影响随机访问场景下单次 fault 的延迟。

5.6.4　性能数据

为验证优化效果，使用 iozone 在 QEMU RISC-V 虚拟机上进行吞吐对比。对比对象为优化前的 StarryOS、优化后的 MossOS（均 1 核 1GB 内存，与竞赛评测环境配置一致），以及 Ubuntu 26.04 基准（4 核 4GB 内存）。

测试场景	优化前	优化后	Ubuntu
顺序写	3524	33221	51922
顺序读	13136	81470	104754
顺序重写	7471	81639	72760
随机读	11082	57668	57175
随机写	7141	59978	59817
反向读	10743	58824	67361
pread 并发读	14184	108348	125270
pwrite 并发写	3724	28263	39335

表 5-4　iozone 性能对比（glibc，单位 kB/sec）

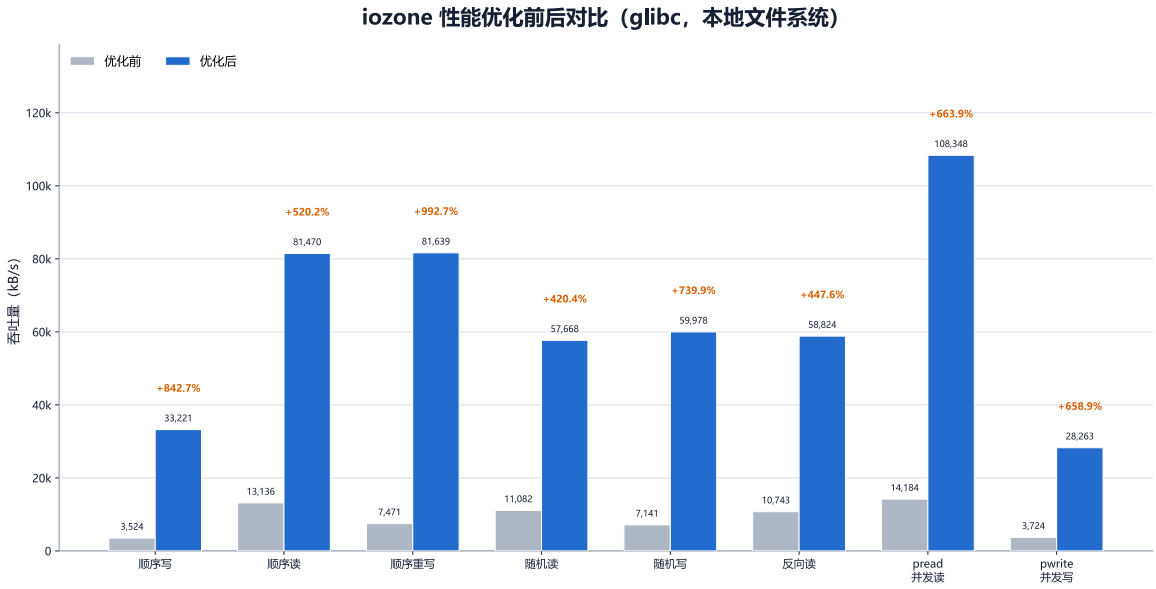


图 5-7　iozone 优化前后吞吐量对比（glibc）

测试场景	优化前	优化后	Ubuntu
顺序写	3524	26407	44818
顺序读	13136	62733	98518
顺序重写	7471	62254	69316
随机读	11082	43813	91905
随机写	7141	51344	57648
反向读	10743	54217	78861
pread 并发读	14184	88315	141988
pwrite 并发写	3724	28192	49303

表 5-5　iozone 性能对比（musl，单位 kB/sec）

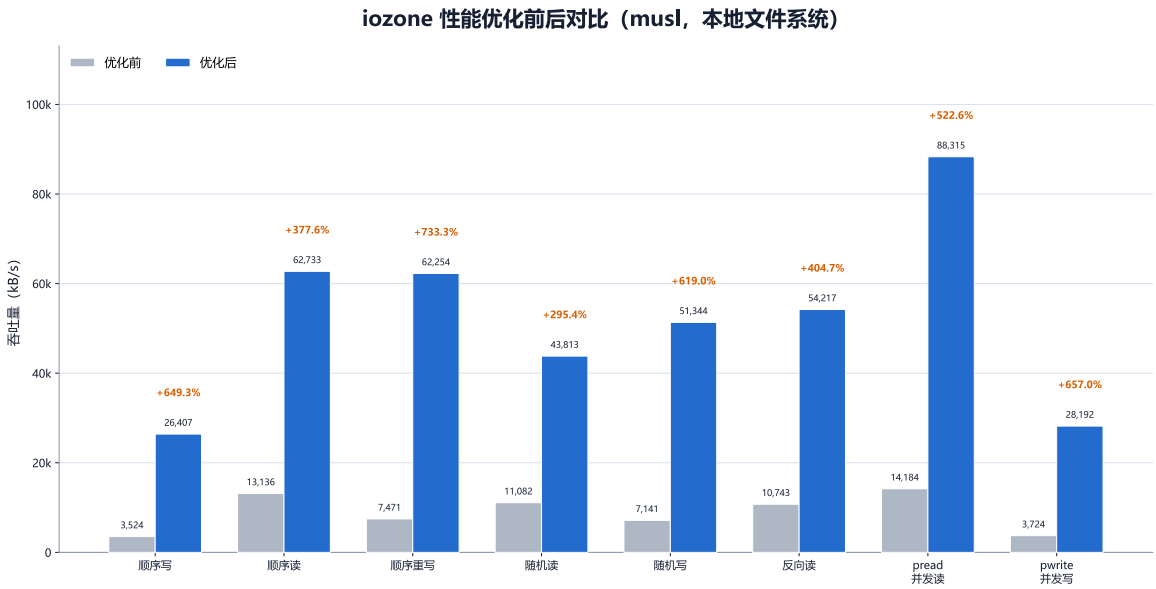


图 5-8　iozone 优化前后吞吐量对比（musl）

综合两组 libc 结果，MossOS 在所有测试项上均显著优于优化前的 StarryOS。随机读写吞吐接近 Ubuntu 基准，顺序重写已在 glibc 环境下超出 Ubuntu。

5.7　本章小结

本章介绍了 MossOS 在 StarryOS 文件系统框架上的六项增量工作。挂载子系统从 tmpfs 扩展到多种文件系统，并补充了 bind/move/remount 及新式 API；inode 元数据缓存将频繁的 stat 和时间戳更新从“穿透底层”变为“内存命中”，并采用 relatime 和延迟提交策略减少元数据写；扩展属性从无到有三层实现，覆盖 12 个系统调用和 ext4 透传；inotify 和 fanotify 填补了文件事件通知的空白，无监听者时通过全局计数和兴趣掩码零开销过滤。I/O 性能优化——全局索引页缓存、避免 read-before-write、脏页合并写回、预读补全——将这些设计选择转化为 iozone 上 5–13 倍的吞吐提升。

第 6 章 信号机制

信号是进程间传递异步事件的基本手段。进程之间可以用信号通知对方，内核也借助信号把硬件异常、终端中断、子进程状态变化等事件交给进程处理。

MossOS 沿用 StarryOS 的信号框架，重点补了三块缺口：进程的可暂停状态、信号发送的权限校验，以及父进程对子进程停止与继续事件的感知。

6.1 StarryOS 的信号机制

6.1.1 整体结构

StarryOS 的信号机制由底层库 `starry-signal` 提供核心语义，内核负责将其接入任务管理、系统调用以及返回用户态的处理路径。信号管理器分两层组织：

- 进程级：保存共享的待处理信号队列、信号处理动作表，以及进程内所有线程的引用。信号发给进程时，从中挑一个没有屏蔽该信号的线程来处理。
- 线程级：保存线程私有的待处理信号、屏蔽集和信号栈。

标准信号只保留一份，重复发送会合并；实时信号按发送顺序排队，每次发送都不丢。

6.1.2 信号检查与递送

每次从内核返回用户态之前都会检查信号。内核从队列里取出一个未屏蔽的信号，根据处理动作决定后续行为——终止、忽略、停止，或者跳到用户注册的处理函数。

调用用户处理函数时，内核在用户栈上构造信号栈帧，保存被中断时的寄存器状态和恢复所需的上下文，再把控制流转过去。函数执行完后，通过 `sigreturn` 系统调用回到内核，内核恢复栈帧继续原来的执行流。

`kill`、`tkill`、`tgkill`、`sigaction`、`sigprocmask`、`sigsuspend`、`sigtimedwait` 等系统调用都建立在这条链路上，MossOS 沿用了这些系统调用的实现。

6.1.3 现存问题

StarryOS 的实现存在三个明显缺口：

1. 停止与继续没有实现。收到 `SIGSTOP`、`SIGTSTP` 时直接让进程退出，对 `SIGCONT` 不做处理。Shell 的作业控制因此完全无法工作。
2. 信号发送没有权限校验。普通用户理论上只能向同 UID 的进程发信号，这个检查在 StarryOS 里缺失。
3. 父进程看不到子进程的停止与继续。`waitid` 的 `WSTOPPED`、`WCONTINUED` 标记形同虚设。

后面几节分别说明 MossOS 如何处理这三个问题。

6.2 进程可暂停状态

6.2.1 问题所在

StarryOS 里进程只有两条路：继续运行，或者退出。`SIGSTOP` 走的是退出路径，但 POSIX 标准规定停止信号应该让进程暂停、保留现场，等 `SIGCONT` 到来后再从原位置继续——停止不等于死。

Shell 按下 Ctrl+Z 时最能说明这个问题。前台进程收到 `SIGTSTP`，应该暂停；用户之后执行 `fg`，进程从断点继续。StarryOS 的实现里 Ctrl+Z 会直接杀掉进程，作业控制整个语义因此垮掉。GDB 等调试器的暂停和恢复依赖的也是同一套机制。

6.2.2 核心结构

要支持暂停，进程需要一个对外可见的“暂停中”状态，以及一个让线程真正挂住自己的地方。MossOS 引入 `ProcessWaitStatus` 枚举，并在 `ProcessData` 上新增两个字段：

```
pub enum ProcessWaitStatus {
    Stopped(Signo),   // 被停止信号挂起，记录触发的信号编号
    Continued(Signo), // 被继续信号唤醒，记录触发的信号编号
}

pub struct ProcessData {
    // ...
    // 父进程通过 waitid 可见的暂停/继续状态
    pub wait_status: SpinNoIrq<Option<ProcessWaitStatus>>,
    // 被停止的线程在此事件上挂起，等 SIGCONT 或 SIGKILL 唤醒
    pub stop_event: Arc<PollSet>,
}
```

代码 6-1　进程可暂停状态的核心结构

`wait_status` 是对外暴露的状态，父进程通过 `waitid` 读到的就是这里；`stop_event` 是线程真正挂住自己的地方，停止后阻塞在这个事件上，等到被唤醒。

6.2.3 处理路径

信号检查路径里，停止和继续两条分支现在变成了：

```
SignalOSAction::Stop => {
    // 写入对外可见的停止状态
    *thr.proc_data.wait_status.lock() =
        Some(ProcessWaitStatus::Stopped(signo));
    // 通知父进程可以从 waitid 取结果了
    wake_waiting_parent(&thr.proc_data);
    // 线程在 stop_event 上挂起，等待被唤醒
    wait_for_continue(&thr.proc_data);
}
SignalOSAction::Continue => {
    *thr.proc_data.wait_status.lock() =
        Some(ProcessWaitStatus::Continued(signo));
    wake_waiting_parent(&thr.proc_data);
    // stop_event 上的线程在信号发送路径里被唤醒
}
```

代码 6-2　停止与继续信号的处理路径

停止路径的顺序是：写状态、通知父进程、线程挂在 `stop_event` 上。继续路径：更新状态，通知父进程，同时唤醒 `stop_event`。`SIGKILL` 同样会唤醒 `stop_event`，确保被挂住的进程能走完退出流程，不会卡死在暂停状态里。

图 6-1 展示了进程在运行、暂停和僵尸三种状态之间的完整流转——停止与继续在运行和暂停之间切换，退出后进入僵尸状态，父进程调用 `wait` 完成最终回收。

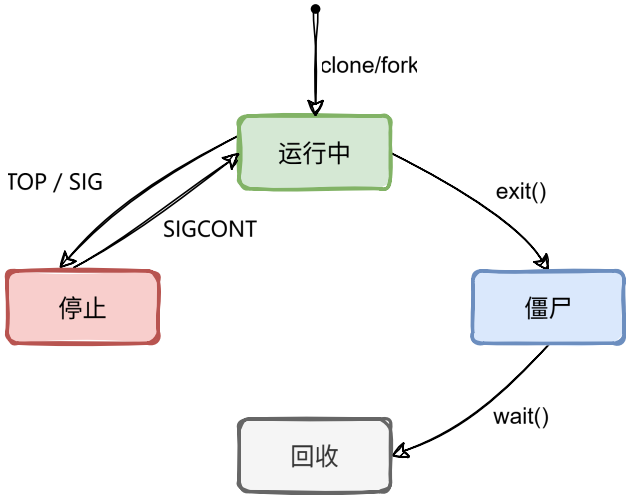


图 6-1　进程状态机

6.3 信号驱动的 wait 语义

进程停止和恢复是信号引起的状态变化，但父进程能否观察到这些变化，取决于 `wait` 系列调用的配置。`waitpid` 和 `waitid` 默认只报告退出事件，要让父进程感知停止与继续，需要额外的选项：

选项	作用
`WUNTRACED`	允许报告子进程停止状态，否则停止事件对父进程不可见
`WCONTINUED`	允许报告子进程继续运行状态
`WNOHANG`	当前没有可报告状态时立即返回，不阻塞父进程
`WNOWAIT`	读取状态但不消费，后续仍可再次查询

表 6-1　子进程等待的主要选项

还是以 Shell 为例。Ctrl+Z 让前台进程组暂停，Shell 需要知道这些进程是“暂停等继续”还是“已经退出”——没有 `WUNTRACED`，Shell 区分不了这两种情况。`WCONTINUED` 处理对称的一侧：`fg` 发出 `SIGCONT` 后，这次状态变化同样需要 `WCONTINUED` 才能被父进程感知到。

如 图 6-2 所示，停止、继续和退出三类事件都可以成为一次 `wait` 的报告。前两者是状态变化通知，父进程读完后子进程仍然存在；退出才是最后一次报告，内核读完后才回收僵尸进程。

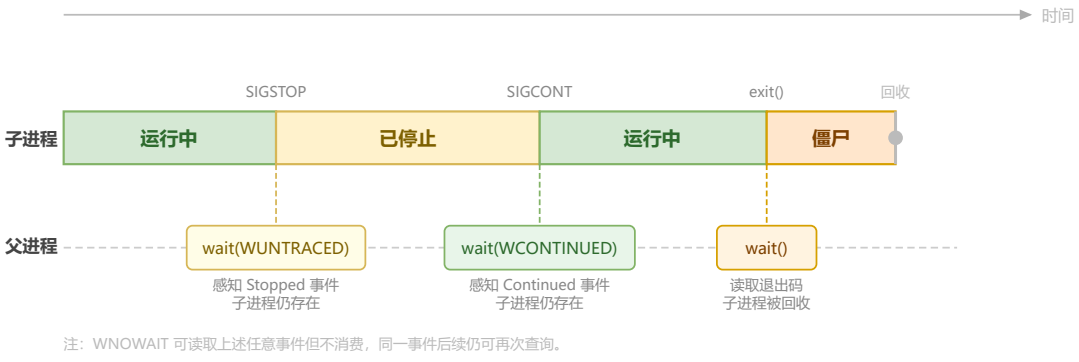


图 6-2　子进程停止、继续、退出三类事件与父进程 wait 消费的时序

`waitid` 相比 `waitpid` 的好处是返回结构化的 `siginfo_t`，能直接区分正常退出、信号终止、停止和继续，不需要调用者手动解析整数状态。`waitid` 还支持通过 pidfd 选择目标——pidfd 是

指向进程对象的文件描述符，不会随 PID 复用而改变指向，配合 `WNOWAIT` 可以先查看状态、之后再完成回收。父进程没有可报告状态时如何进入阻塞、状态变化后怎样被唤醒，见下一节。

6.4 父进程如何感知状态变化

6.4.1 通知路径

子进程状态变化发生在子进程自己的执行流里，父进程可能正阻塞在 `waitid` 上。这里需要一套机制：状态变化时，主动唤醒正在等待的父进程。

StarryOS 已有一个 `child_exit_event`，但只在子进程退出时触发。停止和继续没有对应的唤醒动作，所以 `WSTOPPED`、`WCONTINUED` 的语义一直没法生效。

MossOS 没有新建通道，而是让停止和继续两条路径也触发同一个 `child_exit_event`，把它的语义扩展为“子进程任何状态变化都触发”：

```
fn wake_waiting_parent(proc_data: &ProcessData) {
    if let Some(parent) = proc_data.proc.parent()
        && let Ok(parent_data) = get_process_data(parent.pid())
    {
        parent_data.child_exit_event.wake();
    }
}
```

代码 6-3　子进程状态变化时唤醒父进程

停止路径和继续路径在写完 `wait_status` 之后都调用这个函数（见 代码 6-2）。父进程被唤醒后重新读 `wait_status`，就能拿到子进程的最新状态。

6.4.2 防止通知丢失

父进程登记唤醒入口（Waker）和子进程状态变化之间有一个竞争窗口：父进程检查完 `wait_status`、还没把 Waker 挂上去，子进程的状态恰好先变了，这次通知就悄悄丢了，父进程会一直等下去。

MossOS 用登记后复查覆盖这个窗口：

```
match check_children().transpose() {
    Some(result) => Poll::Ready(result),
    None => {
        // 暂无结果，登记父进程的 Waker
        child_exit_event.register(cx.waker());
        // 登记完立刻再查一次，覆盖登记窗口里发生的状态变化
        match check_children().transpose() {
            Some(result) => Poll::Ready(result),
            None => Poll::Pending,
        }
    }
}
```

代码 6-4　父进程等待子进程状态，登记后复查防止通知丢失

第一次检查没结果，把 Waker 登记上去，然后立刻再查一次。如果第二次查到了结果，说明状态变化恰好落在那个窗口里，直接返回不需要等待。如果仍然没有，才放心地进入阻塞。这套双重检查的通用模式在第三章阻塞与唤醒机制里有更完整的说明。

6.5 进程身份与信号权限校验

POSIX 规定，除 root 外，发送者只能向与自己 UID 或 SUID 匹配的目标发信号。`SIGCONT` 有个例外：同一会话内可以跨 UID 发，让 Shell 能对已切换身份的子进程发出恢复命令。

StarryOS 的 `ProcessData` 里没有 UID 相关字段，检查根本没有基础。MossOS 在进程与线程管理一章的 UID/GID 三元组一节引入了 real/effective/saved 三元凭据，信号权限校验直接使用这些字段：

条件	允许发送
发送者有效 UID 为 0（root）	是
发送者 UID 或 EUID 与目标 UID 或 SUID 匹配	是
发送 `SIGCONT` 且与目标处于同一会话	是
其余情形	否

表 6-2　信号发送的权限规则

`sys_kill` 支持四种目标语义：指定进程、当前进程组、广播、指定进程组。批量操作的结果有三种情况：至少有一个目标有权限就算成功，全部因权限不足失败才返回 `EPERM`，完全没有匹配目标则返回 `ESRCH`。

6.6 小结

StarryOS 已经完成了信号发送、入队、屏蔽与用户处理函数递送的完整链路。MossOS 在此基础上补了三个环节：

补充内容	核心结构	支撑的使用场景
进程可暂停	`ProcessWaitStatus`、`stop_event`	Shell 作业控制、GDB 调试暂停
信号驱动 wait 语义	`WUNTRACED`、`WCONTINUED` 选项生效	父进程按需感知子进程停止与恢复
状态变化通知	`child_exit_event` 复用，登记后复查	`waitid` 停止与继续事件正常到达父进程
信号权限	UID/EUID/SUID 三元凭据校验	POSIX 信号权限模型，跨 UID 隔离

表 6-3　MossOS 对信号机制的补充

第 7 章 网络模块

网络模块负责组织网络数据的收发与传输。MossOS 基于 StarryOS 改造而来，网络栈的整体框架沿用了 StarryOS 已有的设计。StarryOS 基于 smoltcp 协议栈实现了 TCP/UDP socket 的基本功能，MossOS 在此基础上针对若干性能瓶颈与兼容性缺口进行了补充。本章先介绍 StarryOS 网络栈的整体结构与现存问题，随后说明 MossOS 所做的增量。

7.1 StarryOS 的网络栈

7.1.1 整体结构

StarryOS 基于开源协议栈 smoltcp 构建网络模块。smoltcp 是一个轻量级的 TCP/IP 实现，专为嵌入式和裸机环境设计，无需堆分配，代码量较小。StarryOS 通过 `axnet` 模块对 smoltcp 进行封装，向上层提供 `socket`、`bind`、`connect`、`send`、`recv` 等接口。设备层支持以太网和 loopback 设备，能够处理 IPv4 流量。

应用程序通过 socket 系统调用发送数据，数据经协议栈封包后由设备发出；接收路径通过轮询设备推进协议栈状态。StarryOS 已完成收发框架的搭建，但在地址族支持、buffer 配置、loopback 优化等方面仍有改进空间。

7.1.2 现存问题

StarryOS 的网络栈在简单 TCP/UDP 通信场景下可以工作，但在复杂负载下暴露出若干问题：

1. 地址族支持不完整。StarryOS 虽然接收 `AF_INET` 和 `AF_INET6` 参数，但内部统一按 IPv4 处理。IPv4-mapped IPv6 地址（如 `::ffff:127.0.0.1`）会导致路由查找失败。iperf 等工具在 glibc/musl 环境下会混用 IPv4、IPv6 和 mapped 地址，基线无法正确处理这些组合。
2. **Socket** 选项语义缺失。缺少 iperf/netperf 等性能测试工具依赖的关键选项。`TCP_INFO` 返回固定假值，`SO_SNDBUF`/`SO_RCVBUF` 不按 Linux 语义翻倍，`IP_RECVERR`、`SO_DONTROUTE` 等返回 `ENOPROTOOPT` 导致应用进入异常路径。
3. **Buffer** 容量偏小。TCP 和 UDP 的发送/接收 buffer 均为 64KB，在高吞吐场景下成为瓶颈。netperf 的 TCP_STREAM 测试仅能达到约 11 Mbits/sec，远低于预期。
4. **Loopback** 路径未优化。本机 `127.0.0.1` 流量仍走完整的外部网络路径，经过多次拷贝和排队。每次 `send()`/`recv()` 都无条件推进整个网络栈，累积的轮询开销显著。

7.2 地址族支持与兼容性补齐

7.2.1 IPv4/IPv6 混用的处理

IPv4 和 IPv6 在 Linux 中并不是完全独立的两套协议。Linux 允许 IPv6 socket 接收 IPv4 连接（通过 IPv4-mapped IPv6 地址），也允许通过 `IPV6_V6ONLY` 选项禁用这一行为。StarryOS 对地址族的处理过于简化，`sys_socket()` 虽然接收不同的 `domain` 参数，但创建的 socket 都按 IPv4 处理。

问题的关键在于地址格式无法匹配。iperf 测试 `::1`（IPv6 loopback）时会创建 IPv6 socket 并传入 IPv6 地址，但 StarryOS 内部按 IPv4 处理，路由查找时发现地址类型不匹配而失败。更复杂的情况是 IPv4-mapped IPv6 地址（`::ffff:127.0.0.1`），它在形式上是 IPv6，但语义上表示 IPv4，需要正确转换才能路由。

MossOS 为 socket 引入显式的地址族标识。在 `TcpSocket` 和 `UdpSocket` 结构中增加 `family` 字段：

```rust
pub struct TcpSocket {
    family: IpFamily,  // 标识 IPv4 或 IPv6
    handle: SocketHandle,
    local_addr: Option<SocketAddr>,
    peer_addr: Option<SocketAddr>,
    ipv6_only: bool,   // 控制 IPv6 socket 是否接受 IPv4-mapped 地址
    // ...
}

pub enum IpFamily {
    Ipv4,
    Ipv6,
}
```

代码 7-1　Socket 中的地址族标识

创建 socket 时根据 `AF_INET` 或 `AF_INET6` 保存对应的 family。Bind 和 connect 操作在接收地址参数时，先检查地址类型是否与 socket family 匹配。对于 IPv6 socket，若地址是 IPv4-mapped 形式且 `ipv6_only` 为 false，则提取其中的 IPv4 部分进行路由；否则按 IPv6 处理。

Listen table 在接收连接时也需配合检查。IPv6 wildcard listener（`::`）在 `ipv6_only` 为 false 时可以接收 IPv4 连接（以 mapped 形式呈现），但在 `ipv6_only` 为 true 时只能接收纯 IPv6 连接。MossOS 在 listen table 的匹配逻辑中增加了 family 和 `ipv6_only` 的判断，确保连接被正确路由到对应的 listener。

7.2.2　Socket 选项的补齐

iperf 和 netperf 等性能测试工具在启动时会设置若干 socket 选项以调整行为。StarryOS 对部分选项的处理不符合 Linux 语义，导致工具进入异常路径或直接失败。MossOS 针对若干关键选项进行了补齐：

选项	MossOS 的处理
`TCP_INFO`	返回包含当前拥塞窗口、RTT、重传计数等信息的结构体，供工具诊断连接状态
`SO_SNDBUF` / `SO_RCVBUF`	按 Linux 语义，用户请求的值在内核中翻倍存储；读取时返回实际分配的值
`TCP_QUICKACK`	控制 TCP 是否立即发送 ACK，影响延迟敏感场景的响应速度
`SO_LINGER`	控制 `close()` 时是否等待未发送数据，以及超时行为
`IP_RECVERR`	允许应用通过 `recvmsg()` 获取 ICMP 错误，iperf UDP 模式依赖此选项
`SO_DONTROUTE`	标记数据包不经过路由表，直接发往本地网络

表 7-1　补齐的 Socket 选项

这些选项的补齐使得 iperf 和 netperf 在 MossOS 上的行为与在 Linux 上保持一致，不再因缺少选项而回退到低性能路径或报错退出。

7.3 Buffer 扩展

StarryOS 原有的 64KB buffer 在高吞吐场景下容易成为瓶颈。发送方或接收方频繁因 buffer 满而阻塞，限制了吞吐上限。MossOS 将 buffer 容量提升至适合高吞吐场景的配置：

```
// TCP buffer 从 64KB 提升到 512KB
const DEFAULT_TCP_RX_BUF_LEN: usize = 512 * 1024;
const DEFAULT_TCP_TX_BUF_LEN: usize = 512 * 1024;

// UDP buffer 从 64KB 提升到 1MB
const DEFAULT_UDP_RX_BUF_LEN: usize = 1024 * 1024;
const DEFAULT_UDP_TX_BUF_LEN: usize = 1024 * 1024;

// Router packet buffer 提升到 256 项
const ROUTER_PACKET_BUFFER_LEN: usize = 256;
```

代码 7-2　Buffer 容量配置

扩展后的 buffer 使得发送方和接收方不再因容量限制而频繁阻塞，吞吐能力显著提升。

7.4 Loopback 路径优化

7.4.1 优化动机

本机流量和外部流量的优化目标完全不同。外部网络需要考虑链路 MTU、拥塞窗口、丢包重传；本机 loopback 是内存内投递，不存在物理链路错误，也不应被网络拥塞算法限制。

StarryOS 的 loopback 流量仍走完整的网络栈路径：send buffer → 协议栈封包 → router queue → loopback device buffer → router RX queue → 协议栈解包，涉及多次拷贝和排队。每次 `send()`/`recv()` 都会无条件推进整个网络栈，即使当前 socket 已经有数据可读写，仍然要遍历所有 socket 并轮询底层设备。对于 loopback 这种纯内存操作，轮询开销占据了大部分 CPU 时间。

7.4.2 Router 直投与大 MTU

MossOS 在 router 层面识别 loopback 目的地址，对这类包直接投递到 RX buffer，跳过设备队列。判断逻辑如下：

```
impl IpAddress {
    pub fn is_loopback(&self) -> bool {
        match self {
            // IPv4: 127.0.0.0/8
            IpAddress::Ipv4(addr) => addr.is_loopback(),
            // IPv6: ::1/128
            IpAddress::Ipv6(addr) => addr.is_loopback(),
        }
    }
}

// Router 发送时判断目的地址
fn route_packet(packet: &Packet) {
    if packet.dst_addr().is_loopback() {
        // 直接投递到 RX buffer, 不经过设备
```

```
            self.rx_buffer.push(packet);
        } else {
            // 外部流量走设备队列
            self.device_queue.push(packet);
        }
    }
```

代码 7-3　Loopback 地址判断与直投

同时，loopback 设备使用独立的 MTU 配置（65535 字节），不受外部网卡 1500 字节限制。大 MTU 减少了分片和重组开销，对大数据块传输的性能提升显著。

7.4.3　TCP 拥塞窗口绕过

TCP 协议的拥塞窗口机制用于防止网络过载，但 loopback 流量不存在网络拥塞。StarryOS 的 smoltcp 对所有 TCP 连接统一应用拥塞控制，loopback 连接的发送速率被人为限制。

MossOS 在 smoltcp 层面为 loopback tuple 添加标记，绕过拥塞窗口检查。发送侧仅受 buffer 容量限制，不再受慢启动和拥塞避免算法约束。这一改动使得 loopback TCP 吞吐从约 11 Mbits/sec 提升至 Gbits/sec 级别。

7.4.4　UDP 直达路径

UDP loopback 的优化更为激进。由于 UDP 无需维护连接状态，MossOS 让 UDP loopback 完全绕过协议栈封包和路由，直接写入目标 socket 的 RX queue：

```
// UDP socket 发送时判断目的地址
fn send_to(&self, buf: &[u8], addr: SocketAddr) -> Result<usize> {
    if addr.ip().is_loopback() {
        // 直接写入目标 socket 的接收队列
        if let Some(target) = find_local_socket(addr) {
            target.rx_queue.push(buf)?;
            return Ok(buf.len());
        }
    }
    // 外部流量走正常协议栈
    self.inner.send_to(buf, addr)
}
```

代码 7-4　UDP Loopback 直达

这一路径省去了封包、路由查找、解包的全部开销，UDP loopback 性能提升数倍。

7.4.5　按需轮询策略

StarryOS 在每次 `send()`/`recv()` 时都会调用网络栈的 poll 函数，遍历所有 socket 并推进协议栈状态。对于已经有数据的 loopback socket，这一轮询是不必要的。

MossOS 将轮询策略改为按需触发。`send()`/`recv()` 先尝试一次真实操作，只有遇到 `WouldBlock`（缓冲区空或满）时才推进网络栈并进入等待。批量操作场景下，多次连续的 `send()` 只在最后一次遇到阻塞时才触发一次 poll，而不是每次都 poll。

`poll()`/`select()`/`epoll_wait()` 等 I/O 多路复用接口在检查 fd readiness 前统一推进一次网络栈，避免每个 socket 重复推进。这一改动在 loopback 场景下减少了约 70% 的 poll 调用次数。

7.5 性能对比

为验证优化效果，使用 iperf3 和 netperf 在 loopback 接口上进行性能测试。所有测试均在本地 QEMU 虚拟机（RISC-V 架构）上运行，单核，1GB 内存。对比优化前后以及 Ubuntu 26.04 基准的性能数据。

7.5.1 iperf3 测试结果

测试场景	优化前 (Mbits/s)	优化后 (Mbits/s)	Ubuntu (Mbits/s)
TCP 基本测试（glibc）	265	1608	3041
TCP 并发测试（glibc）	239	1935	3574
TCP 反向测试（glibc）	283	1618	3277
UDP 基本测试（glibc）	36.4	1167	2908
UDP 并发测试（glibc）	19.6	1260	3840
UDP 反向测试（glibc）	38.9	1280	2744

表 7-2　iperf3 性能对比（glibc，本地 loopback）

表 7-2 给出了 glibc 环境下各测试项的原始吞吐量。图 7-1 进一步用柱状图直接对照优化前后的测试结果，并在柱顶标出相对提升幅度。

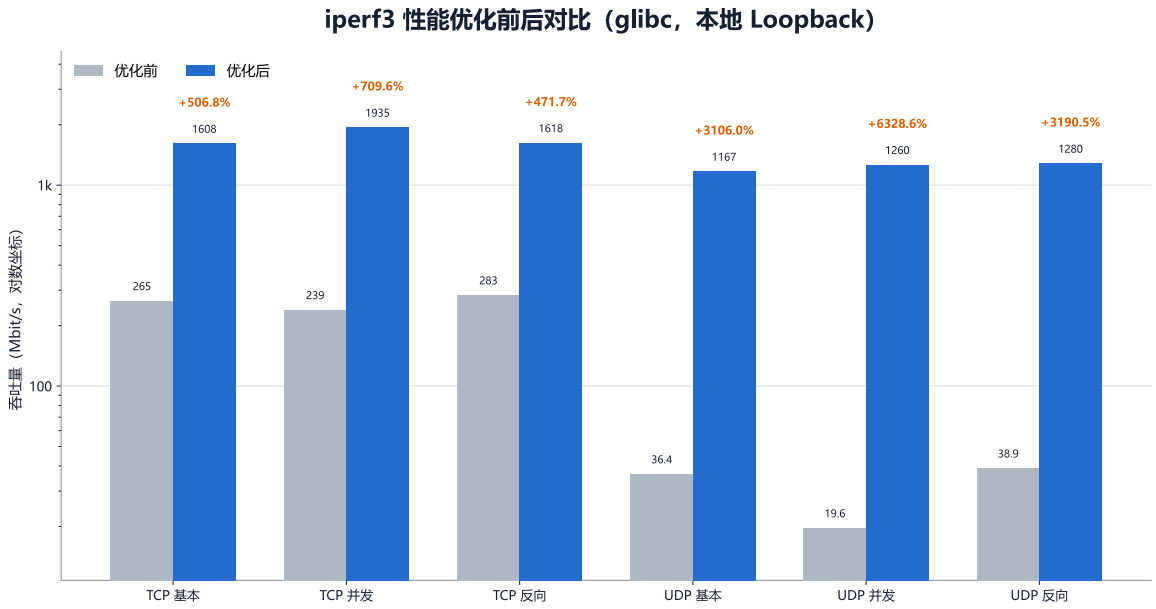


图 7-1　iperf3 原始吞吐量优化前后对比（glibc）

musl 环境采用相同的测试场景和统计口径，原始吞吐量如下。

测试场景	优化前 (Mbits/s)	优化后 (Mbits/s)	Ubuntu (Mbits/s)
TCP 基本测试（musl）	245	1669	3267
TCP 并发测试（musl）	207	1997	2755
TCP 反向测试（musl）	278	1638	3379
UDP 基本测试（musl）	34.7	1178	2324
UDP 并发测试（musl）	23.9	1249	3809
UDP 反向测试（musl）	37.4	1270	2427

表 7-3　iperf3 性能对比（musl，本地 loopback）

图 7-2 使用相同方式展示 musl 结果。可以看到，TCP 和 UDP 各场景均有显著提升，其中 TCP 并发以及 UDP 基本、反向场景的改善尤为明显。

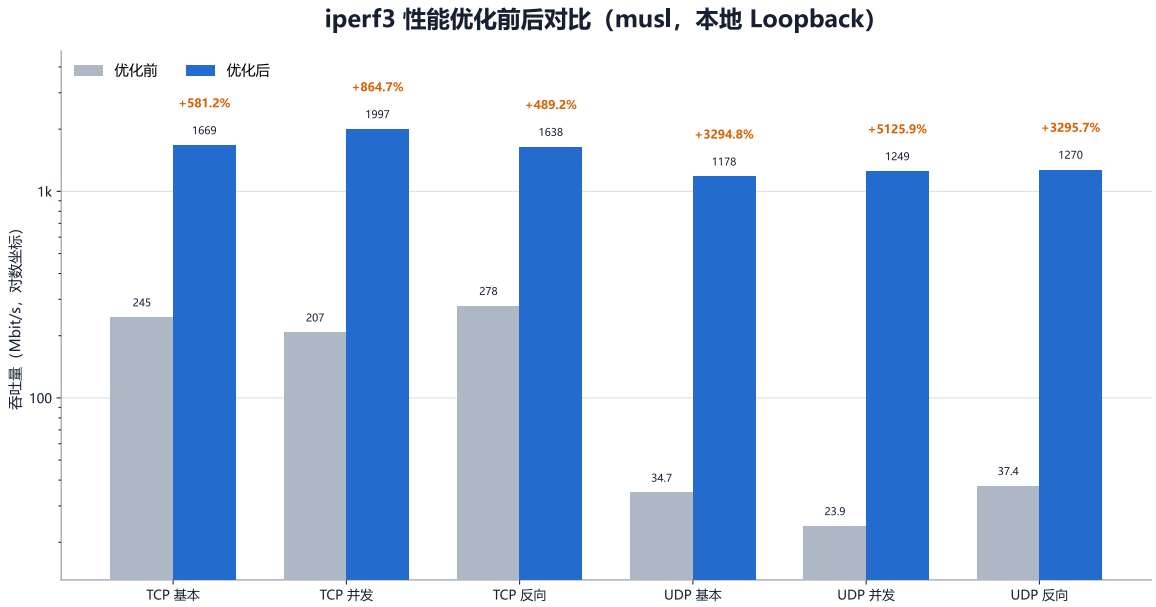


图 7-2　iperf3 原始吞吐量优化前后对比（musl）

7.5.2　netperf 测试结果

iperf3 侧重持续吞吐，netperf 则进一步覆盖流式传输、请求响应和连接建立等不同负载。由于这些子项的原始单位并不完全相同，表格保留各自单位，后续柱状图也按单位分区展示。

测试场景	优化前	优化后	Ubuntu
TCP 流式传输 (Mbits/s)	11.01	299.35	277.37
UDP 流式传输 (传输率/秒)	159.85	192.11	127.63
TCP 请求响应 (传输率/秒)	1591.55	2681.80	2544.46
UDP 请求响应 (传输率/秒)	1644.41	2140.90	2807.66
TCP 连接请求响应 (传输率/秒)	146.30	274.08	753.23

表 7-4　netperf 性能对比（glibc，本地 loopback）

为避免把 Mbits/s 与传输率/秒混在同一坐标轴中，图 7-3 将 TCP_STREAM 单独绘制，其余事务率项目使用另一坐标区，并直接比较优化前后的原始结果。

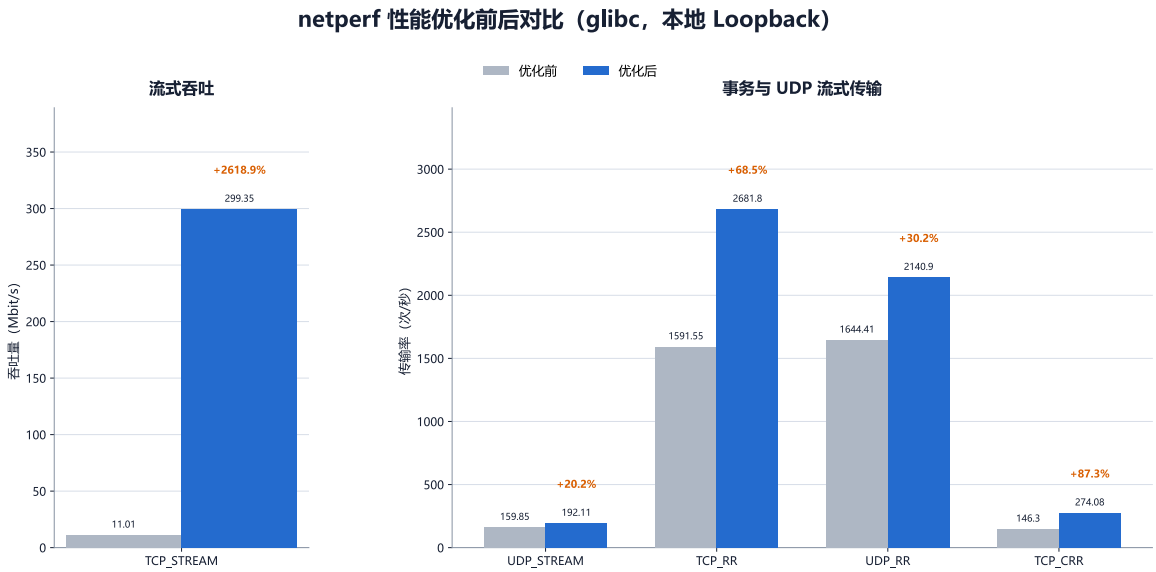


图 7-3　netperf 原始性能优化前后对比（glibc）

综合表格和柱状图可见，优化前的 StarryOS 基线在 loopback 吞吐测试中远低于 Ubuntu 基准，主要原因是 loopback 流量仍走完整网络栈路径且受拥塞窗口限制。优化后，TCP loopback 吞吐提升约 6-27 倍，UDP loopback 吞吐提升约 32-64 倍，已接近或部分超过 Ubuntu 在同等测试环境下的表现。netperf TCP_STREAM 从约 11 Mbits/sec 提升至 299 Mbits/sec，超过 Ubuntu 的 277 Mbits/sec；UDP_STREAM 和 TCP_RR 也超过对应 Ubuntu 参考值，而 UDP_RR 与 TCP_CRR 仍有进一步优化空间。

7.6　小结

StarryOS 已经搭建起基于 smoltcp 的网络栈框架，提供了 TCP/UDP socket 的基本功能。MossOS 在此基础上针对若干性能瓶颈与兼容性缺口进行了补充：

方面	StarryOS	MossOS 增量
地址族	仅 IPv4	区分 IPv4/IPv6，支持 IPv4-mapped IPv6 和 `IPV6_V6ONLY`
Socket 选项	缺少关键选项	补齐 `TCP_INFO`、`SO_*BUF`、`TCP_QUICKACK` 等
Buffer 容量	TCP/UDP 均 64KB	TCP 512KB，UDP 1MB，Router 256 项
Loopback 优化	走完整网络栈	Router 直投、大 MTU、绕过拥塞窗口、UDP 直达、按需轮询

表 7-5　MossOS 对网络模块的补充

这些增量使得 MossOS 能够正确运行 iperf、netperf 等网络性能测试工具。Loopback 吞吐从约 11 Mbits/sec 提升至 Gbits/sec 级别；相关快速路径只对本机地址启用，非 loopback 流量仍经过正常协议栈。

第 8 章 QEMU 虚拟硬件平台与设备驱动

MossOS 在 QEMU 虚拟机上支持 RISC-V 64 和 LoongArch64 两种架构。两者的页表格式、异常编码、中断控制器和设备总线完全不同，StarryOS 已通过三层平台抽象将这些差异封闭在底层实现中，避免调度器、内存管理器和系统调用处理代码散布架构条件分支，MossOS 沿用了这套设计。本章说明该抽象机制的具体实现，以及 VirtIO 如何在两种架构上提供一致的设备接口。

8.1 平台抽象层架构

8.1.1 三层隔离设计

MossOS 将硬件差异分三层处理，每层只吸收它自己那一层的差异。

第一层是 `axcpu`，处理 CPU 本身的寄存器和异常机制。`axcpu/src/riscv/` 实现 RISC-V 的 Trap 上下文保存与恢复、用户态进入与返回；`axcpu/src/loongarch64/` 做同样的事，用 LoongArch 的 `era`/`badv` 等 CSR。两边向上提供相同的 `TrapFrame`、`context_switch`、`uspace` 接口。

第二层是 `axplat`，用 Rust trait 定义平台接口：`IrqIf`（中断注册与分发）、`TimeIf`（时钟读取与单次定时器）、`MemIf`（物理内存区域描述）、`ConsoleIf`（串口输出）。`axplat-riscv64-qemu-virt` 和 `axplat-loongarch64-qemu-virt` 各自实现这些 trait，通过 `#[impl_plat_interface]` 宏完成编译期静态绑定。

第三层是 `axhal`，把 `axplat` trait 调用包装成普通函数，供内核其余部分使用。调度器、内存管理器和驱动框架调用的是 `axhal::time::wall_time()`、`axhal::irq::register()` 这类接口，不直接触碰任何指令集特有寄存器。

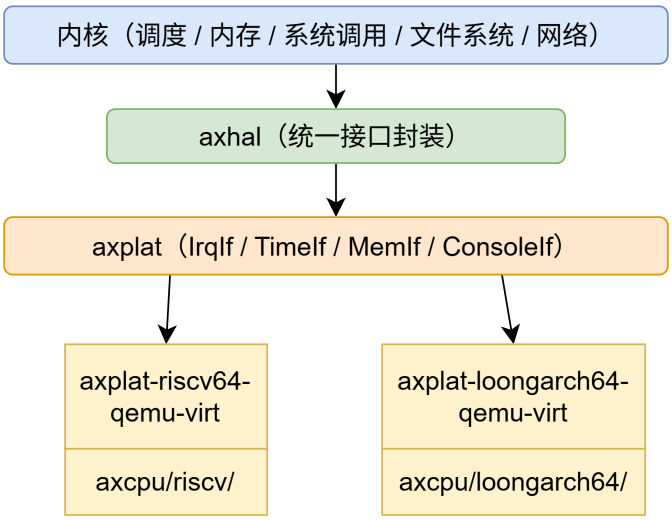


图 8-1　平台抽象三层结构

8.1.2 中断控制器抽象

RISC-V64 的外部设备中断经 PLIC 路由。`scause` 区分定时器（`S_TIMER`）、核间中断（`S_SOFT`）、外部中断（`S_EXT`）三类。处理外部中断时需先向 PLIC claim 获得设备 IRQ 号，处理完后再 complete 确认。

LoongArch64 的路由链不同：设备中断由 PCH-PIC 收集后送入 EIOINTC，再由 EIOINTC 向处理器核心报告；定时器中断通过 `ecfg` 寄存器的 `LineBasedInterrupt::TIMER` 位独立控制。

两套机制都实现了同一个 `IrqIf` trait：

方法	RISC-V64 实现	LoongArch64 实现
`set_enable`	操作 `sie` CSR 位或 PLIC priority/enable 寄存器	操作 `ecfg.lie` 位或 EIOINTC 使能寄存器
`register`	按 `scause` 值分别注册 timer / IPI / PLIC handler	按 `TIMER_IRQ` / `EIOINTC_IRQ` 分类注册
`handle`	外部中断时 claim PLIC 得到真实 IRQ 号，再调度 handler	读 EIOINTC 状态位后调度 handler
`send_ipi`	通过 SBI `send_ipi()` 指定 hart mask 发送	通过 EIOINTC IPI 功能发送

表 8-1　IrqIf 在两种架构下的实现：接口相同，硬件路径不同

驱动框架调用 `axplat::irq::register(irq, handler)` 时，对 PLIC 还是 EIOINTC 一无所知。

8.1.3 时钟接口抽象

RISC-V64 通过 SBI（`sbi_time::set_timer`）设置定时器，`time::read64()` 读取当前计数；LoongArch64 使用 CPU 内置的稳频计数器，`tcfg`/`tval` 寄存器控制定时器触发。两者同样通过 `TimeIf` trait 统一：`current_ticks()` 读各自的硬件计数器，`ticks_to_nanos()` 完成频率转换，`set_oneshot_timer()` 设置下一次中断。调度器的时间片计算和 `clock_gettime` 系统调用只操作 `axhal::time::monotonic_time()`，不感知底层计数器差异。

8.2 双架构 QEMU 平台对比

资源	RISC-V64 QEMU virt	LoongArch64 QEMU virt
启动方式	OpenSBI，ELF 内核直接加载	UEFI BIOS，ELF 内核直接加载
页表格式	Sv39 三级页表，`satp` CSR	LoongArch 四级页表，`PGDL`/`PGDH` CSR（见第 4 章）
中断控制器	PLIC，MMIO `0x0c00_0000`	PCH-PIC + EIOINTC
定时器	SBI timer，`mtime`/`mtimecmp`	稳频计数器，`tcfg`/`tval`
设备总线	VirtIO MMIO（固定地址范围枚举）	VirtIO over PCIe（ECAM + BAR 配置）
串口	8250 UART，MMIO	16550 UART，MMIO

表 8-2　两种 QEMU 平台的关键硬件资源对比

页表格式的差异已在第 4 章展开，本章不重复。

8.3 VirtIO 设备驱动

8.3.1 两条发现路径，一个驱动入口

赛方两种平台均提供两块 VirtIO 磁盘（`x0` 承载文件系统，`x1` 为额外测试磁盘）和一张 VirtIO 网卡，区别仅在于设备挂载的总线不同。

RISC-V64 上，QEMU virt 机型将 VirtIO 设备寄存器映射到固定 MMIO 地址范围。`axdriver` 初始化时遍历该范围，将物理地址转换为内核虚拟地址后调用 `probe_mmio_device()` 探测设备类型。

LoongArch64 上，VirtIO 设备挂在 PCIe 总线上。`axdriver` 通过 PCI ECAM 枚举 BDF，匹配厂商 ID `0x1af4`，按设备 ID 区分块设备（`0x1001/0x1042`）和网卡（`0x1000/0x1041`），配置 BAR 后调用 `probe_pci_device()` 初始化传输层。

两条路径都以 `VirtIoTransport` 为参数进入 `D::try_new(transport, irq?)`，其中 `D` 是实现了 `VirtIoDevMeta` 的具体类型（`VirtIoBlk` 或 `VirtIoNet`）。成功初始化的设备通过 `AxDeviceEnum` 注册到全局设备表，文件系统挂载和网络协议栈直接取用，不感知底层总线差异。

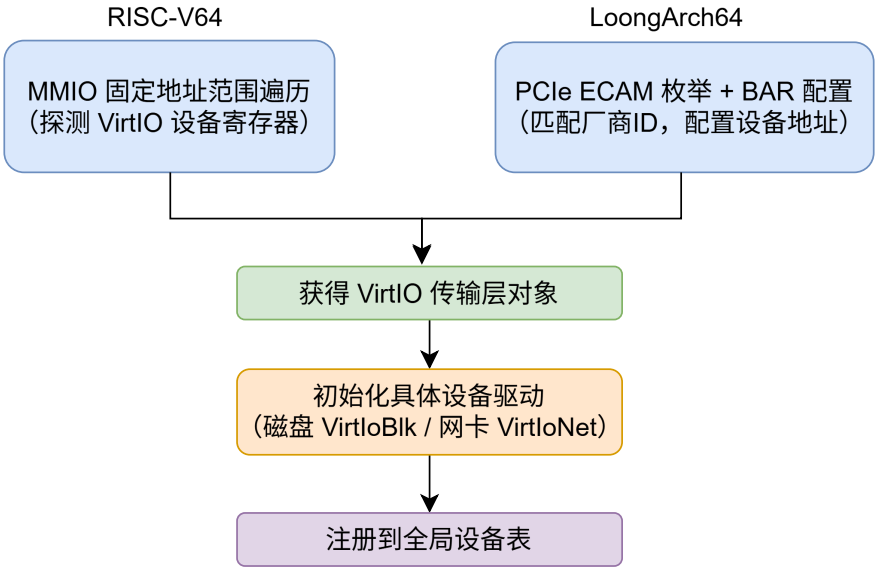


图 8-2　VirtIO 设备发现路径

8.3.2 VirtIO 队列与 DMA

VirtIO 协议以 virtqueue 为数据通道：驱动将控制信息和数据缓冲区组织为描述符链，写入可用队列（available ring）并通知 QEMU；QEMU 完成后将结果写入已用队列（used ring），驱动据此回收描述符。块设备用它提交扇区读写，网卡用它收发报文，两者共用同一套队列管理模型。

VirtIO 描述符中的缓冲区地址必须是物理地址。`VirtIoHalImpl` 实现 `VirtIoHal` trait，负责 DMA 页面的分配与地址转换：

```
// axdriver/src/virtio.rs
unsafe impl VirtIoHal for VirtIoHalImpl {
    fn dma_alloc(pages: usize, _dir: BufferDirection) -> (PhysAddr,
NonNull<u8>) {
        let Ok(vaddr) = global_allocator().alloc_pages(pages, 0x1000,
UsageKind::Dma)
        else { return (0, NonNull::dangling()) };
        let paddr = virt_to_phys(vaddr.into());
        (paddr.as_usize(), NonNull::new(vaddr as _).unwrap())
    }
    unsafe fn dma_dealloc(_paddr: PhysAddr, vaddr: NonNull<u8>, pages: usize) -
> i32 {
        global_allocator().dealloc_pages(vaddr.as_ptr() as usize, pages,
UsageKind::Dma);
        0
    }
```

```
    unsafe fn mmio_phys_to_virt(paddr: PhysAddr, _size: usize) -> NonNull<u8> {
        NonNull::new(phys_to_virt(paddr.into()).as_mut_ptr()).unwrap()
    }
}
```

代码 8-1 `VirtIoHalImpl` 的 DMA 分配与地址转换，两种架构共用同一份实现

`dma_alloc` 从全局分配器申请物理连续内存并将物理地址填入描述符；请求结束后 `dma_dealloc` 释放这些页面，避免设备访问已回收的缓冲区。MMIO 寄存器通过 `mmio_phys_to_virt` 映射后访问。这套接口在 RISC-V64 和 LoongArch64 上完全相同。

8.4 跨平台行为一致性

axplat trait 是 ISA 差异的终止线。内核核心模块只调 `axhal` 的接口，不直接操作 PLIC 或 EIOINTC，不读取 `mtime` 或 LoongArch 稳频计数器，也不关心 VirtIO 设备在 MMIO 还是 PCIe 总线上。平台实现层把硬件差异翻译成统一接口，差异到此为止，不再向上传递。

赛方为两种架构提供相同的设备拓扑，内核在两种平台上经历相同的初始化序列和 I/O 路径。用户态进程看到的文件系统语义、网络行为和时钟语义不因底层 ISA 而不同。

第 9 章 总结与展望

9.1 工作总结

MossOS 目标是在 RISC-V 64 和 LoongArch64 上提供接近 Linux 的系统调用语义，使部分 Linux 应用程序能够不加修改地运行。

在进程与执行模型上，MossOS 支持 fork/clone/exec/exit/wait 的主要路径，包括线程组、进程组和会话的三层组织，以及 UID/GID 凭据管理和 pidfd 接口。调度器支持 CPU 亲和性和部分 rlimit 资源约束；futex 实现了 WAIT/WAKE、REQUEUE 和 robust list，能够支撑 glibc 的 pthread 库。

在内存管理上，MossOS 完善了 mmap/munmap/mremap/mprotect 的主要路径，支持匿名映射、文件映射和写时复制，缺页处理覆盖了懒分配和 COW 触发两条路径。文件系统方面，MossOS 在 StarryOS 的 VFS 框架基础上补全了挂载子系统（bind/move/remount 及 fsopen/fsconfig/fsmount 新式 API），引入了 inode 元数据缓存以减少 stat 和时间戳更新的开销，实现了扩展属性（xattr）和文件事件通知（inotify、fanotify），并通过页缓存增强、写路径优化和预读补全将 I/O 吞吐提升了 5–13 倍。信号机制覆盖标准信号与实时信号的主要语义，包括权限校验、停止/继续状态、嵌套信号帧和基础 signalfd。

网络方面，MossOS 支持 TCP、UDP 和 Unix Domain Socket，实现了 socket、bind、connect、accept、send、recv 及相关选项接口。设备驱动通过三层平台抽象屏蔽架构差异，支持 virtio 块设备、网卡和串口，两种目标架构共享同一套内核源码。

9.2 未来工作

当前实现覆盖了宏内核的主干功能，但仍有若干方向值得在决赛阶段继续推进。

在功能扩展方面，io_uring 是 Linux 5.1 引入的高性能异步 I/O 接口，通过共享环形缓冲区避免系统调用开销，对 I/O 密集型负载有显著收益，希望以后能够参考 io_uring 实现一套异步 IO 机制。seccomp 系统调用过滤机制是容器安全的基础，也是运行沙箱类应用的前提条件。mount namespace 和 user namespace 目前只有基础实现，完整的容器隔离语义需要进一步完善。

在性能优化方面，当前调度器采用 RR 策略，对交互型和批处理型任务缺乏区分；引入多级反馈队列或 CFS 风格的调度器可以改善实际负载下的响应延迟。粗粒度锁在多核场景下是主要瓶颈，页缓存、文件描述符表和进程树的锁粒度均有细化空间。